Reconstruction Order, Not Solver Family, Governs Control Accuracy: A Comparison of Neural and Classical HJB Solvers Against an Exact Reference

Jingwen Zhang
School of Mathematics and Physics
Xi'an Jiaotong-Liverpool University
Suzhou, China

Fei Ma
School of Mathematics and Physics
Xi'an Jiaotong-Liverpool University
Suzhou, China

***Abstract*—Value accuracy does not bound control accuracy. On the Hamilton–Jacobi–Bellman (HJB) equation of optimal execution, the reconstruction that turns nodal values into a gradient can dominate the error in the strategy actually executed. With a multilinear reconstruction the control error is 3.2 to 26 times the value error. Replacing it with a cubic reconstruction on the same nodes raises the control-error order from one to about 2.4, cutting the control/value error ratio from 13.6 to 1.7, and costs nothing at the grid level. Two propositions fix the rates rather than merely observing them: a degree-k reconstruction delivers order k — capped at 2α if the gradient is probed at step h^α, so a probe that shrinks only like h caps a cubic at order two — and, for this problem, the objective lost is *exactly* the R-weighted squared control deviation accumulated along the executed path. What governs control accuracy is therefore the order of the reconstruction, not the choice of solver family.**

This reframes the choice between neural and classical solvers, which is what we set out to measure. Physics-informed neural networks (PINNs) are widely proposed for optimal control in finance on the grounds of the curse of dimensionality, but it is unclear when a neural solver is the right choice and when a classical one is. We compare twelve solver configurations across five families: explicit finite differences, an implicit splitting scheme and Howard policy iteration on the grid; a direct physics-informed network and a policy-iteration variant of it; the deep BSDE scheme and its modern relative, deep splitting; and regression Monte Carlo, a classical mesh-free method that the earlier version of this work had only cited. The benchmark is a multi-asset optimal-execution HJB whose value function has an exact matrix Riccati reference, both without a predictive signal and with one. Because that reference is exact rather than another solver, it removes the circularity of scoring one solver against another, and it stays exact at every state dimension — so accuracy can be measured where no grid can run.

Against that reference the comparison gives a boundary rather than a winner. Where a grid scheme can run ($d \le 5$) it is both faster and more accurate, by a factor of 5.6 to 114 in time at $d = 4$ depending on whether the scheme is compiled. Beyond that the classical side does not disappear: regression Monte Carlo, which needs no grid either, also runs, and on this benchmark it is more accurate than the neural solver at every dimension tested — R^2 of 0.999 at $d = 12$ and 0.977 at $d = 20$, against the neural solver's 0.927 and 0.757, in seconds on a CPU rather than minutes on a GPU. Multilevel Picard, the classical method with a complexity theory behind it, reaches 0.26% relative error at $d = 20$ in 2.3 seconds where no grid can run. That last result holds at a wider spread: at the benchmark's own calibration the Picard iteration multilevel Picard realises is not a contraction and it diverges, with the boundary measured. A single term — the maximised Hamiltonian, whose coefficient is $1/\eta$ — sets the grid's time-step limit, destabilises the explicit deep-splitting recursion, and breaks that contraction. So the mesh-free region is not the neural solver's alone, and the accuracy it offers there decays without any internal signal that it is decaying. On the paper's own market calibration the reconstruction result is worth 2.7 to 8.4 basis points of notional across nine instruments. The neural solver's failure at high dimension is a property of *parameterisation*, not of dimension: the same architecture fails on a symmetric problem and succeeds on a non-separable one at matched state dimension.

***Index Terms*—Hamilton–Jacobi–Bellman equation, physics-informed neural networks, neural operators, optimal execution, numerical methods, solver benchmarking, method selection.**

I. Introduction

Deep learning methods for partial differential equations (PDEs) are widely proposed for optimal control in finance on the grounds that mesh-based solvers suffer from the curse of dimensionality. In high-frequency trading (HFT) the relevant problem is the Hamilton–Jacobi–Bellman (HJB) equation of optimal execution, whose state is the trader's inventory and, when a predictive signal is modelled, the signal that drives it. The natural question is practical rather than rhetorical: should such a problem be solved with a neural solver or a classical one, and does the answer change with dimension? This paper answers by running both against an answer that is known exactly, both without a signal and with one.

The comparison is worth making carefully. Much of the applied literature scores a solver against *another solver*, which makes the reference exact by construction and its reported error zero; a closed-form solution removes that circularity. Most comparisons also report one accuracy number, the value function, when the quantity an HJB is solved *for* is the control — a derivative, obtained differently by each family and, as it turns out, not equally accurately. And most are run in a single regime, whereas the answer here depends on the regime.

This paper compares two solver families on a benchmark whose answer is known in closed form, at matched accuracy, scored on *both* the value function and the control. Section IV turns the comparison into rates: two propositions give the order a degree-k reconstruction can deliver, the probe step it requires, and the objective lost when the control is wrong. Three findings follow. First, the comparison yields a boundary rather than a winner: a grid scheme is faster and more accurate wherever it is runnable ($p \leq 5$), and beyond it the mesh-free methods take over — where the classical one among them, regression Monte Carlo, turns out to be the more accurate of the two we measure, at a fraction of the cost. Second, and more usefully, value accuracy does not bound control accuracy: the control error is governed by the order of the reconstruction the grid scheme differentiates, not by the accuracy of the solution, so a cubic reconstruction on the same nodes raises that order from one to between two and three, for free — worth 2.7 to 8.4 basis points of notional on the paper's real-market calibration. Third, the neural solver's failure is a property of the problem's parameterisation rather than its dimension. We close with a decision guide: which regime favours which method, and the measurement that decides it.

II. Related Work

Three literatures meet at this paper's question: how the optimal-execution problem is *formulated*, how it has been *solved*, and what the recent mesh-free turn has changed. The formulation is settled; the solver choice is where the question lies open.

Formulation. Avellaneda and Stoikov derived closed-form approximations for market-making control under inventory risk [1]. Guéant *et al.* formulated optimal execution under market impact as a nonlinear HJB problem [2], and Guéant extended the closed-form approximation to several assets [3]. Cartea and Jaimungal added risk criteria and microstructure features [4], and Almgren and Chriss gave the canonical mean–variance trajectory [5], which is the forced-liquidation limit of our benchmark and is treated at book length in [6]. Everything this paper measures sits inside that formulation; the coefficients are synthetic, and we return to what that costs us in Section VII.

Solving it. The classical treatments are monotone finite-difference schemes [7] and policy iteration [8], both accurate and well understood but scaling poorly in the number of state variables. Their convergence theory is Kushner and Dupuis [9], and the dynamic-programming background for the value iteration we compare against is [10]. The poor scaling is the stated motive for the neural turn: PINNs embed the PDE residual in the loss [11], alongside the deep Galerkin and deep Ritz methods [12], [13] and deep BSDE solvers [14], [15]. Within that last family the variants differ in how the control is represented and in what is regressed: the deep BSDE scheme of [16], the deep splitting method [17], and the Malliavin-weighted formulations of [18], with [19] surveying the stochastic-control setting. We measure the plain deep BSDE scheme of [14], [15] in Table XI, so the family is represented by its reference member rather than by its strongest variant [20]–[22] – a distinction worth stating, since the variants were introduced precisely to address the training behaviour we report. High-order derivatives are the recognised bottleneck [23], and domain decomposition and adaptive refinement reduce training cost [24], [25].

The mesh-free turn, and the two gaps it leaves. The strongest precedents for *scale* are mesh-free and single-instance: the deep Galerkin method reached a 21-dimensional HJB, and a penalty formulation of the coupled HJB–continuity system solved mean-field games in 100 dimensions [26]. Those are arguments for mesh-free representation, which we do not dispute. The same representation choice is now standard for hedging and control directly rather than for the value function [27], a neighbouring problem with the same cost question and no classical baseline. Learned solution *operators* are the more recent line: FNO [28], [29], CNO [30], Transolver [31] and others map function spaces rather than solving one instance. Their cost claims are framed at *inference* time, and that framing is legitimate rather than a strawman: the classical baseline must be re-solved for every new instance. FNO states this directly, and UGrid concedes it from the other side [32]. That is why benchmark work reports training and inference cost separately [33], and why this paper reports both. No paper in that corpus applies an operator to HJB, optimal control or Bellman equations, so the gap addressed here is the absence of an accuracy-matched, exactly-referenced cost comparison at HFT-relevant dimensions.

Two methodological gaps follow, and they set this paper's design. First, most comparisons score a solver against *another solver*, which makes the reference exact by construction and its reported error zero. Removing that circularity needs an external exact solution, and the optimal-execution HJB has one in closed form. Second, most comparisons report a single accuracy number, the value function, when what an HJB is solved *for* is the control. The control is a derivative, each family obtains it differently, and it is not obtained equally accurately — which is the finding we lead with, and which no comparison design built on a single accuracy number can detect. One classical family is absent from the main comparison and we now run it separately. Backward dynamic-programming methods that avoid a mesh by simulating paths and regressing the continuation value — the least-squares Monte Carlo family [34], treated at book length in [35], and its stochastic-control variants [36], [37] — reach high dimension without a grid, and they are the natural third family to place against the two we measure. Section VI-E runs them on the same problem and the same reference. It was the right thing to do and it changed a conclusion: the mesh-free region is not the neural solver's alone, and on this benchmark the classical regression method is the more accurate of the two. The remaining comparison we still do not make is against a tuned, modern deep-BSDE variant rather than its reference member, and we say so wherever that number is quoted. Finally, the SHoP framework [38] has been applied to this HJB in earlier work. We do not claim SHoP, or any solver here, as

a new method: the findings below concern the choice between existing families rather than the introduction of another.

III. Benchmark Problem and Analytic Reference

Let $x \in \mathbb{R}^d$ denote the state and $u \in \mathbb{R}^m$ the trading rate, with linear dynamics and additive noise,

$$dx = (Ax + Bu)\,dt + \Sigma\,dW. \tag{1}$$

The trader maximises expected risk-adjusted proceeds,

$$\sup_u \mathbb{E}\left[-\tfrac{1}{2}\int_0^T \big(x^\top Q x + u^\top R u\big)dt - \tfrac{1}{2}x_T^\top Q_T x_T\right], \tag{2}$$

where $u^\top R u$ is temporary market impact ($R = \eta I$), $x^\top Q x$ is an inventory risk penalty ($Q = \lambda\sigma^2 I$), and the terminal term penalises residual inventory. These coefficients are set synthetically rather than calibrated to an instrument: the object here is the behaviour of solvers on a problem with a known answer, and a calibrated η and an empirically estimated bid–ask spread would change the numbers without changing the comparison. The standard estimators for that calibration step, if it is wanted, are Roll's implicit spread [39] and the high–low estimator of Corwin and Schultz [40]. In the inventory-only case the state is the inventory itself, $x = q \in \mathbb{R}^p$ across p assets, so $d = m = p$; with $A = 0$, $B = I$ this is the Almgren–Chriss problem in LQ form and reproduces the classical trajectory. The HJB equation is

$$-\partial_t V = \max_u \Big\{ -\tfrac{1}{2}\big(x^\top Q x + u^\top R u\big) + (\nabla V)^\top (Ax + Bu)\Big\} + \tfrac{1}{2}\operatorname{tr}\big(\Sigma\Sigma^\top \nabla^2 V\big), \tag{3}$$

with $V(x,T) = -\frac{1}{2}x^\top Q_T x$. Substituting the quadratic ansatz $V(x,t) = -\frac{1}{2}x^\top P(t)x + c(t)$ gives the matrix Riccati equation and an additive constant,

$$-\dot P = Q + PA + A^\top P - PBR^{-1}B^\top P, \qquad P(T) = Q_T, \tag{4}$$

$$\dot c = \tfrac{1}{2}\operatorname{tr}\big(\Sigma\Sigma^\top P\big), \qquad c(T) = 0, \tag{5}$$

and the optimal control

$$u^\star(x,t) = -R^{-1}B^\top P(t)\,x. \tag{6}$$

Eq. (6) is what makes the control testable without a sign convention to argue about: in the inventory-only case $P(t) \succeq 0$, so a liquidation problem with $q > 0$ has a *negative* optimal rate, and a solver that returns the positive of this quantity is inverted — any backtest built on it trades the wrong way in every period. In the scalar case P has the closed form $P(t) = \eta k\,[Q_T \cosh k(T{-}t) + \eta k \sinh k(T{-}t)]/[\eta k \cosh k(T{-}t) + Q_T \sinh k(T{-}t)]$ with $k = \sqrt{Q/\eta}$; as $Q_T \to \infty$ it tends to $\eta k \coth(k(T-t))$ and the optimal trajectory to the classical $q_t = q_0 \sinh(k(T-t))/\sinh(kT)$. We use both limits as independent checks on the reference.

A. A predictive signal

A trader with no view should trade only to manage inventory, so the benchmark above is economically natural but limited. Adding a mean-reverting signal, with $dq = u\,dt$ and $ds = -\kappa s\,dt + \sqrt{2\kappa}\,dW$, and paying the trader for holding inventory in the signal's direction keeps the problem linear–quadratic, so the reference stays exact: the state becomes (q, s), giving $d = 2p$ while $m = p$, and Q becomes *indefinite* rather than positive definite. Two consequences matter here. Separability is broken by construction rather than by randomising coefficients — with perfectly uniform parameters $\|P_{\text{off}}\|/\|P_{\text{diag}}\|$ reaches 0.205, against *exactly* zero in the uniform inventory-only case — and because P is indefinite the control has no fixed sign, so (6) is checked pointwise against the reference instead of against a sign rule. We use $\kappa = 1$ and a signal scale of 0.005, putting the target inventory for a unit signal near 0.5. No elementary closed form survives once $A \neq 0$, so the released code validates this reference by the same route as above.

The state dimension d is free, so the curse-of-dimensionality claim can be measured rather than asserted, and every method solves the same equation, so the comparison isolates representation and discretisation.

IV. What Governs Control Accuracy

The measurements below say that control accuracy is set by the order of the operator that turns stored nodal values into a gradient, and not by the accuracy of the stored values. This section states that as two propositions, because the claim is a rate rather than an observation, and because one of its consequences — how the probe step must be scaled — is what reconciles two tables in this paper that would otherwise appear to disagree.

Fix a mesh of spacing h, let R_h be a reconstruction of degree k (piecewise linear for $k = 1$, cubic for $k = 3$), and let $\widehat{u} = -R^{-1}B^\top \nabla(R_h V)$ be the control a grid solver extracts from it.

Proposition 1 (reconstruction and probe). *Let $V \in C^{k+1}$ and suppose the nodal values are exact. On the interior of a cell,*

$$\big\|\nabla(R_h V)(x) - \nabla V(x)\big\| = O(h^k), \tag{7}$$

and if the gradient is obtained by a symmetric difference of step ε instead of exactly,

$$\big\|D_\varepsilon(R_h V)(x) - \nabla V(x)\big\| = O(h^k) + O(\varepsilon^2). \tag{8}$$

Hence with $\varepsilon = c\,h^\alpha$ the attainable order is $\min(k, 2\alpha)$; the reconstruction's own order k survives only if $\varepsilon = o(h^{k/2})$.

Proof sketch. The first estimate is the standard derivative loss of one order in polynomial interpolation: the interpolant reproduces polynomials of degree k, so the difference $\nabla(R_h V) - \nabla V$ is the gradient of an $O(h^{k+1})$ interpolation error. For the second, D_ε is the average of $\nabla(R_h V)$ over $[x - \varepsilon, x + \varepsilon]$ up to $O(\varepsilon^2)$, so its error is the first error plus the symmetric-difference truncation $O(\varepsilon^2)$; the two are independent and add. □

The practical content is in the exponent: a cubic reconstruction differentiated with a probe that shrinks like h cannot beat order 2, however good the reconstruction is. Table VI measures this — the same cubic reaches 2.2–2.8 at $\varepsilon = h/4$ and 3.4–3.8 at $\varepsilon \ll h$ — and `exp_probe_scaling.py` sweeps the exponent and recovers 2.10–2.37 at $\alpha = 1$, 3.32–3.79 at

$\alpha = 1.5$ and 3.36–3.83 at $\alpha = 2$, against the predicted 2 and 3 and the reconstruction's own ceiling of ≈ 3.4.

Proposition 2 (control error and objective loss). *For the linear–quadratic benchmark, let u be any admissible control and x the state it produces from x_0, let $u^\star$ and $x^\star$ be the optimal feedback and its path, and write $\delta u_t = u(x_t, t) - u^\star(x_t, t)$. Then*

$$J(u^\star) - J(u) \;=\; \tfrac{1}{2}\,\mathbb{E}\left[\int_0^T \delta u_t^\top R\,\delta u_t \; dt\right] \tag{9}$$

exactly, with no remainder term. Consequently (i) *the objective loss is the R-weighted squared control deviation accumulated along the path actually taken, so a degree-k reconstruction — control error $O(h^k)$ — loses $O(h^{2k})$ of objective;* (ii) *the position error obeys $\|x_t - x_t^\star\| \le T\|B\| \sup_t \|\delta u_t\|$, so the path inherits the control error at first order; and* (iii) *the loss is governed by the control error and not by the value error, which bounds neither.*

Proof. Write the verification discrepancy of the HJB at an arbitrary control, $D(x,u) = V_t + (\nabla V)^\top (Ax + Bu) + \frac{1}{2}\mathrm{tr}(\Sigma\Sigma^\top \nabla^2 V) - \frac{1}{2}(x^\top Q x + u^\top R u)$. Substituting $V = -\frac{1}{2}x^\top P(t) x + c(t)$ and the Riccati equation cancels every quadratic term in x and leaves

$$D(x,u) = -\tfrac{1}{2}\big(u - u^\star(x,t)\big)^\top R\big(u - u^\star(x,t)\big), \tag{10}$$

with $u^\star(x,t) = -R^{-1}B^\top P(t)x$. Dynkin's formula gives $J(u) = V(x_0,0) + \mathbb{E}\int_0^T D(x_t,u)\,dt$, and $D(x,u^\star) \equiv 0$; subtracting the same identity at $u^\star$ gives (9). The bound in (ii) is Gronwall on $d(x - x^\star) = (A(x-x^\star) + B(u-u^\star))dt$. □

Part (i) is the reason the proposed remedy is worth its cost: raising the reconstruction from first to third order moves the loss from $O(h^2)$ to $O(h^6)$, and it is free at the grid level. It also replaces an appeal to the envelope theorem, which would give only that the loss is second order in the deviation, with an identity that says which deviation and along which path. `verify_suboptimality_identity.py` checks (9) numerically: with an exact reference the two sides agree to 4×10^{-10} relative, and the residual is shown to be the checker's own piecewise-linear interpolation of $P(t)$, falling as $n_{\text{ref}}^{-1.9}$ as that grid is refined. The same check confirms the bound in (ii) in every case, and shows the measured orders in Section VI-C: the loss converges at 2.1 where (9) plus a first-order control error predicts 2.

One caveat belongs with the identity, and it is the one place where a low-order extraction is worse than its order suggests. A multilinear reconstruction's gradient is piecewise *constant*, so the extracted control is discontinuous in x and the closed-loop dynamics are only piecewise smooth; with a cubic reconstruction, whose gradient is continuous, the same numerical check agrees to 9×10^{-9} instead of 7×10^{-5}. The asymmetry is in the trajectory integration rather than in the identity, but it is a real additional cost of the piecewise-constant control.

A. Monotonicity: when a high-order reconstruction must be limited

Proposition 1 says to raise the order. Convergence theory for viscosity solutions says something that pulls the other way: the Barles–Souganidis framework requires the scheme to be *monotone*, consistent and stable [41], [42], and a high-order reconstruction of ∇V is not monotone. Whether that conflict binds depends on where the gradient enters the scheme, and this problem separates the three cases cleanly.

(i) Post-processing. Here the Hamiltonian maximiser is available in closed form, $u^\star = -R^{-1}B^\top P(t)x$, and is substituted into the residual analytically. The value function is therefore solved by a scheme in which the reconstruction does not appear at all, so its monotonicity is untouched, and the reconstruction acts only on the control that is read out afterwards. This is why the order result of Proposition 1 can be had without reopening the convergence argument, and it is a property of the benchmark rather than a general one.

(ii) Policy iteration. The extracted gradient defines the frozen policy, which does enter the evaluation operator, so the reconstruction now influences the scheme. It does not follow that monotonicity is lost: monotonicity is a property of how the *frozen* advection is discretised, not of how the policy was obtained. Our implementation upwinds it for exactly this reason — an earlier version that central-differenced a frozen advection term was unconditionally unstable and diverged near $t = T$, which is how the requirement was discovered here. What the reconstruction changes is the fixed point, not the monotonicity of the step that finds it.

(iii) A general HJB. When $u^\star$ is not available in closed form, ∇V enters the Hamiltonian discretisation itself and the scheme's monotonicity does depend on how it is approximated. There the conflict is real and is the reason essentially non-oscillatory reconstructions exist [43]–[45]; monotone families that sidestep it exist too — semi-Lagrangian schemes are the standard example [46] — at the cost of a different restriction on the time step. It is also worth saying plainly that raising the order of a recovered derivative is not itself a new idea — gradient recovery and post-processing have a long history in finite elements [47], [48], and the error of a discretised control is classical in PDE-constrained optimisation [49], [50]. What is specific to this paper is which of the two errors a practitioner is buying, and that the control can be wrong in *direction* while the value is right.

That last possibility is measurable, and Table I measures it. The unlimited cubic is compared with the same gradient passed through a monotonised-central limiter built from the nodal one-sided slopes — a post-limiter, not a different solve. On smooth data with exact nodal values the limiter is inert: no wrong-way samples either way, and the same order (3.41). Where the data have an interior turning point the clipping costs order ($3.72 \to 1.51$) and still buys nothing. The picture inverts once the nodal data are not smooth. Across a gradient kink the unlimited cubic overshoots the true gradient by $1.33\times$, and on nodal values carrying 2% noise — the realistic case, since the

TABLE I

ORDER AGAINST MONOTONICITY, ON FUNCTIONS WHOSE GRADIENTS ARE KNOWN. *cubic* IS THE UNLIMITED RECONSTRUCTION OF PROPOSITION 1 WITH A PROBE FAR BELOW THE MESH; *limited* IS THE SAME GRADIENT PASSED THROUGH A MONOTONISED-CENTRAL LIMITER BUILT FROM THE NODAL ONE-SIDED SLOPES. *Wrong way* IS THE FRACTION OF SAMPLES WHERE THE RECONSTRUCTION HAS THE OPPOSITE SIGN TO A SIGNIFICANT TRUE GRADIENT — FOR A CONTROL, WHICH IS A DERIVATIVE, THAT IS A TRADE IN THE WRONG DIRECTION RATHER THAN A SMALL ERROR, AND RETURNING ZERO AT A TURNING POINT IS THE SAFE ANSWER AND IS NOT COUNTED. *Overshoot* IS THE LARGEST RECOVERED GRADIENT RELATIVE TO THE LARGEST TRUE ONE. THE ORDER COLUMN IS GIVEN ONLY WHERE AN ORDER EXISTS: A DISCONTINUOUS GRADIENT AND NOISY DATA HAVE NONE. ON SMOOTH DATA THE LIMITER IS INERT (NO WRONG-WAY SAMPLES EITHER WAY) AND COSTS NOTHING EXCEPT AT INTERIOR TURNING POINTS, WHERE CLIPPING DROPS IT FROM 3.7 TO 1.5. ON DATA THAT ARE NOT SMOOTH IT IS WHAT STOPS THE CONTROL POINTING THE WRONG WAY.

	wrong way (%)		overshoot (×)	
data condition	cubic	limited	cubic	limited
smooth, monotone	0.0	0.0	1.00	1.00
smooth, interior turning point	0.0	0.0	1.01	1.00
gradient kink	0.2	0.0	1.33	1.24
2% nodal noise (solved grid)	8.6	0.0	2.26	1.64
convergence order, where one exists				
smooth, monotone: cubic 3.41, limited 3.41				
smooth, interior turning point: cubic 3.72, limited 1.51				

reconstruction is applied to a *solved* grid and not to exact data — it points the wrong way at 8.6% of samples and overshoots by $2.26\times$. The limited reconstruction never points the wrong way in any of these cases, at 0% throughout.

The recommendation is therefore conditional, and the condition is checkable before any solve: raise the reconstruction order, and limit it where the nodal data are not monotone. For this benchmark and every smooth instance of it the limiter is unnecessary; for a value function with a kink, a constraint, or a boundary layer, the unlimited reconstruction is the one that will trade the wrong way.

V. METHODOLOGY

The four methods fall into two families on the one axis this paper is about. A grid scheme stores V at N^d nodes and must *reconstruct* a continuous function from them before it can differentiate one; a network *is* a continuous function, so automatic differentiation returns its gradient exactly. Everything the paper reports about control accuracy follows from that difference, and from the order of the reconstruction the grid scheme uses.

Solvers. The *finite-difference* scheme uses explicit backward time stepping on a tensor-product grid of N points per dimension, with the analytic Hamiltonian maximiser and central differences in space. *Policy iteration* uses Howard's algorithm on the same grid: policy evaluation solves the linear frozen-policy PDE and policy improvement sets $u = R^{-1}B^\top \nabla V$. Two implementation points matter. Because the frozen-policy term is genuine advection, the evaluation step must be upwinded — central differencing of a frozen advection term is unconditionally unstable. And the policy must be stored as a function of (x, t), because $P(t)$ is strongly time-dependent; collapsing it to a single array made the evaluation PDE inconsistent near $t = T$.

The PINN. The network $\mathcal{N}$ maps $(x, t) \mapsto V$ and is trained by minimising the HJB residual

$$r(x,t) \;=\; \frac{\partial V}{\partial t} \;+\; \frac{1}{2}\sigma^2 \Delta V \;-\; \tfrac{1}{2}\big(x^\top Q x + u^{\star\top} R u^\star\big) \;+\; \big(\nabla V\big)^\top \big(Ax + Bu^\star\big), \tag{11}$$

with $u^\star = R^{-1}B^\top \nabla V$ substituted analytically, so the supremum over the control never has to be learned. Three design choices carry the accuracy. The terminal condition is imposed *exactly* rather than as a penalty, by writing

$$V(x,t) \;=\; V_T(x) \;+\; (T-t)\,\mathcal{N}(x,t), \tag{12}$$

so $V(x, T) = V_T(x)$ for any network parameters and no penalty weight has to be tuned against the residual; this matters more here than in a forward PDE, because the terminal region is where the control is largest. The residual is divided by its value at initialisation, so the loss starts at $O(1)$ and the same learning rate works at every p, and collocation points are drawn with density proportional to $(1 - u)^\beta$ in time ($u \sim U(0, 1)$, $\beta = 3$), concentrating samples near $t = T$ where $|\nabla V|$ is largest. Finally the inputs are scaled by an explicit component — the state box affinely to $[-1, 1]$, time to $[0, 1]$ — which is not cosmetic: it is the component whose omission produces the out-of-domain failure of Section VII-C. The architecture is a 4×64 $\tanh$ MLP trained with Adam [51]. *No architecture search is performed*: the network is fixed and is not tuned per dimension, so every neural statement here is a claim about a competently trained standard PINN rather than about the best available one. Section VI-F then measures how much that choice matters.

Protocol and configuration. All metrics are computed against the analytic solution, never against another solver; value and control are reported separately because they turn out not to agree (Section VI-C); and three reconstructions of the grid solution are compared — a piecewise-linear (multilinear) one probed by finite differences, that same reconstruction differentiated exactly, and a cubic one probed identically. The defaults ($\eta = 0.01$, $\lambda\sigma^2 = 0.01$, $Q_T = 0.1$, $\sigma = 0.01$, $T = 1$) give $k = 1$ and keep the control magnitude $O(1)$–$O(10)$, so the explicit time step is set by a realistic Courant–Friedrichs–Lewy (CFL) bound rather than by an artificial stiffness. Neural training and timings run on a single NVIDIA RTX 4090 Laptop GPU (17.2 GB) and grid timings on the CPU. That asymmetry would make a wall-clock comparison unfair on its face, so the cost results are reported twice: as measured seconds, and as an analytic count of floating-point work with the rate each family achieves on the hardware it ran on Section VI.

VI. RESULTS

Below, the dimension and cost results use the inventory-only benchmark, where $d = p$; the signal extension of Section III-A is used wherever a result could be an artefact of that model.

TABLE II

ACCURACY AND COST BY STATE DIMENSION, AGAINST THE CLOSED-FORM RICCATI SOLUTION. THE GRID COLUMNS USE $N^\star$, THE SMALLEST RESOLUTION REACHING VALUE $R^2 \geq 0.9999$ AT THAT DIMENSION (FOUND BY REFINEMENT), SO THE TWO FAMILIES ARE COMPARED AT EQUAL ACCURACY RATHER THAN EQUAL RESOLUTION. PINN COLUMNS ARE A *fixed* BUDGET OF 6000 EPOCHS AND 2048 COLLOCATION POINTS; A LARGER BUDGET RESCUES $p = 4$ AND IS DISCUSSED IN THE TEXT. TIMES ARE GPU FOR THE PINN AND CPU FOR THE GRID SCHEME. A NEGATIVE R^2 MEANS THE LEARNED VALUE FUNCTION IS WORSE THAN PREDICTING THE MEAN, I.E. THE SOLVER HAS FAILED RATHER THAN MERELY LOST ACCURACY.

	grid scheme (accuracy-matched)				PINN (fixed budget)			
p	$N^\star$	time (s)	value R^2	ctrl R^2	time (s)	value R^2	ctrl R^2	sign agr.
1	21	0.023	0.99995	0.9958	95.1 ± 4.3	0.9997 ± 0.0002	0.9995 ± 0.0001	0.989 ± 0.007
2	21	0.056	0.99993	0.9957	128.2 ± 5.4	0.9993 ± 0.0003	0.9989 ± 0.0005	0.985 ± 0.008
3	21	0.530	0.99993	0.9958	156.6 ± 10.3	0.9918 ± 0.0109	0.9955 ± 0.0026	0.985 ± 0.005
4	21	31.710	0.99992	0.9959	177.1 ± 1.0*	-57.0218 ± 46.5787	-0.1707 ± 0.7816	0.996 ± 0.001
5	21	864.873	0.99991	0.9959	225.9 ± 4.4*	-615.7841 ± 0.6823	-1.6490 ± 0.0000	1.000 ± 0.000

*PINN accuracy has collapsed ($R^2 < 0.99$).

A. *Accuracy and cost against the analytic solution*

Table II is the central result of the paper.

Before the results, the scope of the comparison should be unambiguous, because a table with two columns invites the reading that two methods were tried. Twelve solver configurations across five families were measured, and Table III lists all of them with the operating point each was measured at. The operating points differ by design rather than by accident: the two families in Table II are compared at *matched accuracy*, which is the only way to ask which is cheaper for the same answer; the neural variants are compared at matched *budget*, so that the number of policy iterations is the variable rather than the compute; and every row shares the same problem and the same exact reference. Three of the rows exist only because of that audit, and two of them changed what the paper says. The implicit scheme shows that the CFL restriction is removable but that on this benchmark it is set by the optimal control rather than by the diffusion, so implicitness buys a factor of four rather than the orders it buys on diffusion-dominated problems. The PI-PINN sweep shows that the second neural variant the abstract refers to needs its iterations resolved: at matched budget, two outer iterations are worse than predicting the mean ($R^2 = -0.15$) while eight recover 0.9997, so the honest statement about that family is conditional on a hyperparameter that the direct variant does not have. And regression Monte Carlo is the row that keeps the mesh-free region from being the neural solver's alone.

Figure 1 is the same comparison as a single picture: accuracy against solve time, with the accuracy-matched grid configuration marked, so the crossing between the two families can be read off rather than assembled from columns. The finite-difference scheme is the most accurate method at every dimension, with value R^2 between 0.99927 and 0.99999, and its control has the correct sign at every sampled state. Policy iteration converges in 13 outer iterations and is slightly less accurate ($R^2 = 0.9908$ to 0.9966), as expected from the first-order upwind discretisation used in its policy-evaluation step.

The cost comparison does not depend on the hardware split. Both families were timed on the machine they run well on — the network on the GPU, the grid sweep on the CPU

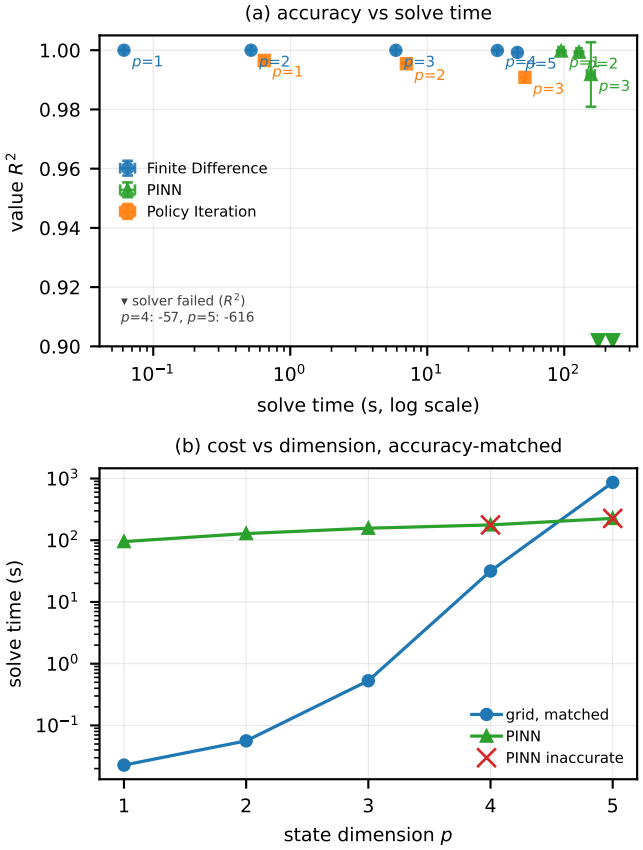


Fig. 1. Accuracy against solve time, both families on the machine each runs well on, with the accuracy-matched grid configuration ($N^\star = 21$) marked. Off-scale PINN failures are clipped and annotated rather than dropped, so a reader can see where the learned solver stops solving the problem instead of inferring it from an absence.

— so we also counted work rather than seconds. Counting $3d^2 + 3m^2 + 2dm + 4d + 2$ floating-point operations per grid point per time step, and $(6+2d)P$ per sample for the network ($P \approx 1.3 \times 10^4$ parameters for training and $2P$ per query), the grid scheme converts work into time at 0.13 to 0.27 GFLOP/s while the network reaches 26 to 30 GFLOP/s: a ratio of about 10^2. Counting work instead of seconds therefore *flatters the*

TABLE III

EVERY SOLVER MEASURED IN THIS WORK, WITH THE OPERATING POINT EACH WAS MEASURED AT. THE HEADLINE COMPARISON OF TABLE II ISOLATES THE TWO FAMILIES THAT CAN BE COMPARED AT *matched accuracy*; THIS IS THE FULL SET, AND THE OPERATING POINTS INTENTIONALLY DIFFER — MATCHED ACCURACY WHERE A GRID CAN RUN, MATCHED *budget* AMONG THE NEURAL VARIANTS SO THAT ITERATIONS RATHER THAN COMPUTE IS THE VARIABLE, THE SAME PROBLEM AND THE SAME EXACT REFERENCE THROUGHOUT. THE ACCURACY COLUMN NAMES ITS OWN METRIC: R^2 FOR THE GRID, NEURAL AND REGRESSION ROWS (VALUE / CONTROL), AND THE RELATIVE ERROR OF THE POINT ESTIMATE AT x_0 FOR DEEP BSDE, WHICH IS THE ONLY QUANTITY THAT SCHEME PRODUCES (TABLE XI GIVES ITS DIMENSION SWEEP). POLICY ITERATION RAN AT A FINER GRID THAN THE ACCURACY-MATCHED COLUMN, AND SAYS SO. TWO ROWS HAD BEEN MEASURED AND NEVER REPORTED BEFORE A READER ASKED HOW MANY METHODS THE COMPARISON COVERED: THE IMPLICIT SCHEME, WHOSE $4\times$ STEP HOLDS AT $R^2 = 0.9997$ ON THE SIGNALLED PROBLEM AND AT 0.9966 ON THE INVENTORY-ONLY ONE — ON THIS BENCHMARK THE STABILITY RESTRICTION IS SET BY THE OPTIMAL CONTROL RATHER THAN BY THE DIFFUSION, SO IMPLICITNESS BUYS LESS THAN IT DOES ELSEWHERE — AND THE PI-PINN SWEEP, WHOSE BUDGET-MATCHED RUNS ARE SENSITIVE TO THE ITERATION COUNT: TWO ITERATIONS ARE WORSE THAN PREDICTING THE MEAN. SOURCES: `EXP_GRID_REFINEMENT_MERGED.JSON`, `EXP_DIMENSION.JSON`, `EXP_IMPLICIT_GRID.JSON`, `EXP_PI_PINN.JSON`, `EXP_BSDE_DIMS.JSON`, `EXP_LSMC.JSON`.

solver	family	operating point	accuracy (metric named per row)	time (s)
explicit finite differences	grid	$p = 2$, $N^\star = 21$	0.99993 / 0.9957	0.056
Howard policy iteration	grid	$p = 2$, $N = 41$	0.99543 / 0.9979	7.05
implicit LOD splitting	grid	signalled, $4\times$ the explicit step	0.99965 / 0.9980	0.040 vs 0.134
direct PINN, 4×64	neural	$p = 2$, 6,000 epochs, 2 seeds	0.99993 / 0.9999	241
PI-PINN, 2 outer iterations	neural	$p = 2$, budget-matched, 2 seeds	-0.14820 / 0.2590	245
PI-PINN, 8 outer iterations	neural	$p = 2$, budget-matched, 2 seeds	$+0.99967$ / 0.9995	247
deep BSDE	probabilistic	$p = 4$, point-valued at x_0	3.27% rel. err / –	1537
regression Monte Carlo ($d = 5$)	classical, mesh-free	21 basis functions	0.99947 / 0.9999	1.5
regression Monte Carlo ($d = 20$)	classical, mesh-free	231 basis functions	0.97715 / 0.9853	12.5
deep splitting (neural regression)	deep splitting	$d = 1$, 50 steps	$+0.1627$ / $+0.8642$	44.7
deep splitting, exact quadratic regression	deep splitting	$d = 1$, 50 steps	$+0.9987$ / $+0.9995$	0.1
multilevel Picard ($d = 20$, $\eta = 1$)	probabilistic	point-valued at x_0	0.26% rel. err / –	2.3

neural solver, which means the wall-clock result below is the conservative one and is not an artefact of giving the network a GPU. One caveat ran the other way in an earlier draft and is now measured rather than conceded: the grid figures come from a NumPy implementation whose per-step cost is many small array operations, and Table IV runs the same scheme as a fused parallel kernel at identical work. It is 2.5 to 11.9 times faster, its achieved rate rises from 0.65 to 7.7 GFLOP/s, and the two implementations agree on the solved field to 1.3×10^{-4} relative. Everything below therefore reports both: the NumPy seconds, which are what the paper's other tables carry and which no well-implemented grid solver would take, and what the same scheme costs compiled.

We had to reduce the grid resolution as p grew, so comparing cost at the resolutions the sweep could afford would be unfair. A separate refinement study swept N at each p and found the smallest resolution reaching a fixed accuracy target of $R^2 \geq 0.9999$. That resolution is $N^\star = 21$ at *every* dimension tested, which gives one accuracy-matched grid configuration per p; the grid columns of Table II use it. Grid work grows far faster than the point count N^d suggests. The explicit time step is CFL-limited by the mesh spacing, so work scales closer to N^{p+1}: at $p = 5$, $N = 21$ is 4.1×10^6 points over 420 steps, which is why accuracy-matched finite differences cost nearly ten minutes of NumPy at a dimension the neural solver covered in under four — and why that is 59 s when the same scheme is compiled.

At matched accuracy the grid scheme is faster than the fixed-budget neural solver at every dimension, by $5.6\times$ at $p = 4$ as measured in NumPy and by $114\times$ with the same scheme compiled, and by two to three orders of magnitude at $p \leq 2$. The mesh-free solver's cost is indeed almost flat in p ($95 \rightarrow 226$ s, a factor of 2.4 across five dimensions) against $0.023 \rightarrow 865$ s for the NumPy grid scheme, so the two curves do cross — but at the crossing the neural solver is no longer solving the problem. Its accuracy is competitive at low dimension ($R^2 = 0.9997 \pm 0.0002$ at $p = 1$) and degrades with p, failing outright at $p \geq 4$ under this budget. A negative R^2 means the learned value function is worse than predicting the mean, so the method has not solved the problem rather than merely lost accuracy. Control-sign agreement stays high even where R^2 is negative: the network recovers the *direction* of the optimal control while missing its level, so sign agreement alone is not evidence that an HJB solver works. *That failure is specific to this parameterisation, not to the dimension* — see Section VI-D.

B. Is the comparison an artefact of the quadratic solution?

Every number above is measured on a value function that is a quadratic form, and a second-order scheme reproduces a quadratic almost exactly. That is why the resolution needed to hit the accuracy target came out at $N^\star = 21$ at *every* dimension, which is a property of the solution rather than of the solvers, and it is the first thing a referee should attack. The attack is answerable, and the answer is in Table V.

We keep the solvers and change the solution. Fix a value function that is not a polynomial,

$$V(x,t) = -\tfrac{1}{2}x^\top P(t)x + A_0(T-t)\sin\big(\omega\,\mathbf{1}^\top x\big), \qquad (13)$$

with P the Riccati solution so that $V(x,T) = -\frac{1}{2}x^\top Q_T x$ still holds, and add to the running reward the source s

TABLE IV

THE GRID TIMINGS, SEPARATED INTO THE SCHEME AND THE IMPLEMENTATION. THE SAME SOLVE — SAME GRID, SAME STENCIL, SAME STEP COUNT, CHECKED TO AGREE ON THE SOLVED FIELD TO 1.3×10^{-4} RELATIVE (THE EDGE CLOSURE; THE INTERIOR IS IDENTICAL) — IS RUN BY THE NUMPY SOLVER USED EVERYWHERE ELSE IN THIS PAPER AND BY A FUSED PARALLEL KERNEL (NUMBA; THE ENVIRONMENT HAS NO JAX). THE IMPLEMENTATION FACTOR GROWS WITH THE PROBLEM, FROM $2.5\times$ AT $p = 1$ TO $11.9\times$ AT $p = 4$, AND THE ACHIEVED RATE RISES — $p = 1$: $0.01 \to 0.01$, $p = 2$: $0.14 \to 0.53$, $p = 3$: $0.58 \to 4.41$, $p = 4$: $0.65 \to 7.65$, $p = 5$: $0.67 \to 6.43$ GFLOP/S — WHICH IS WHAT THE FLOP MODEL IN THE TEXT WAS THERE TO CORRECT FOR: THE NUMPY FIGURE WAS MEASURING PER-STEP ARRAY OVERHEAD AS MUCH AS ARITHMETIC. THE CONSEQUENCE FOR THE COMPARISON IS AT $p = 5$, WHERE THE NUMPY GRID WAS SLOWER THAN THE NEURAL SOLVER (865 S AGAINST 226 S) AND THE COMPILED ONE IS FASTER (59 S). BOTH COLUMNS ARE THE TIME-STEPPING SWEEPS ONLY, AT THE SAME FORCED STEP COUNT.

p	N	steps	NumPy (s)	kernel (s)	factor
1	21	420	0.02	0.009	2.5×
2	21	420	0.05	0.015	3.7×
3	21	420	0.58	0.076	7.7×
4	21	420	18.48	1.558	11.9×
5	21	420	567.10	59.252	9.6×

that makes it exact, $s = -V_t + \frac{1}{2}x^\top Qx - (\nabla V)^\top Ax - \frac{1}{2}(\nabla V)^\top BR^{-1}B^\top \nabla V - \frac{1}{2}\mathrm{tr}(\Sigma\Sigma^\top \nabla^2 V)$. Then V solves the HJB of the modified problem and the maximiser is unchanged — s does not depend on u, so $u^\star = -R^{-1}B^\top \nabla V$ is still optimal — which means *the same solver, the same stencil and the same CFL bound* apply, with one added term. That matters: a reimplementation would have answered a question about the reimplementation. (Ours did, first time round: it froze the boundary values instead of using the solver's one-sided stencils and went unstable before producing a number.)

What changes is the physics of the question. The same accuracy target, relative L^2 value error $\leq 10^{-3}$, is now *not reached at* $N = 61$ *in any dimension*, where the quadratic benchmark reached it at $N = 21$ with room to spare ($R^2 \geq 0.9999$). The dimension-free resolution was therefore an artefact of the quadratic solution. Three things are worth separating in the rest of the table. The value error still converges at the scheme's order (2.0–2.2), so the grid method is not degraded by the non-polynomial term — it simply has more to resolve. The control error converges at 1.04–1.13, which is Proposition IV's prediction for a piecewise-linear reconstruction and not an artefact of the quadratic either. And the cubic reconstruction keeps a nine- to seventeen-fold advantage over the multilinear one *on the same nodes*, so the paper's headline result survives where a cubic reconstruction is no longer exact on the solution.

Two honest qualifications. First, the resolution requirement did not turn out to grow with dimension in the way the objection anticipates: with $\omega = 3$ the dominant error is the CFL-tied time step, which is dimension-independent, so the failure to reach the target is a statement about resolution rather than about dimension. Push the non-polynomial term to $\omega = 12$ — amplitude-matched, so the control magnitude and the CFL bound are unchanged, and only the spatial scale moves — and the error at $N = 61$ rises to 1.2×10^{-2} at $d = 1$ and 5.1×10^{-3} at $d = 2$, with apparent orders of 2.8–3.1 as the error crosses from the under-resolved spatial

TABLE V

THE BENCHMARK'S DIMENSION-FREE RESOLUTION IS A PROPERTY OF ITS SOLUTION, NOT OF THE SOLVERS. THE QUADRATIC ROWS ARE THE PAPER'S BENCHMARK, WHOSE VALUE FUNCTION IS A QUADRATIC FORM: A SECOND-ORDER SCHEME REPRODUCES IT ALMOST EXACTLY, AND $N^\star = 21$ SUFFICES FOR THE ACCURACY TARGET AT EVERY DIMENSION. THE NON-POLYNOMIAL ROWS REPLACE IT WITH A MANUFACTURED EXACT SOLUTION, $V = -\frac{1}{2}x^\top P(t)x + 5 \times 10^{-3}(T - t)\sin(3 \cdot \mathbf{1}^\top x)$, MADE EXACT BY A SOURCE IN THE RUNNING REWARD, AND SOLVED BY *the same solver* WITH THE SAME STENCIL AND CFL BOUND. THE TARGET (RELATIVE L^2 VALUE ERROR $\leq 10^{-3}$) IS THEN NOT REACHED BY $N = 61$ IN ANY DIMENSION. THE CONTROL-ERROR ORDER AND THE CUBIC-VS-MULTILINEAR GAIN ARE GIVEN AT THE FINEST MESH: THE ORDER STAYS NEAR ONE, AS PROPOSITION 1 PREDICTS FOR A PIECEWISE-LINEAR RECONSTRUCTION, AND THE CUBIC RECONSTRUCTION KEEPS A NINE- TO SEVENTEEN-FOLD ADVANTAGE ON THE SAME NODES — SO THE PAPER'S HEADLINE RESULT DOES NOT DEPEND ON THE SOLUTION BEING A POLYNOMIAL.

	$d = 1$	$d = 2$	$d = 3$
quadratic solution (this paper's benchmark)			
$N^\star$ for the target: 21, 21, 21			
non-polynomial solution (manufactured, same solvers)			
value rel. L^2 at $N = 61$	1.6e-03	2.0e-03	2.3e-03
target 10^{-3} reached	no	no	no
value order vs N	2.16	2.11	2.03
control order vs N	1.13	1.08	1.04
cubic gain at $N = 61$	9.3×	16.6×	15.5×
the same, with the non-polynomial term under-resolved ($\omega = 12$)			
value rel. L^2 at $N = 61$	1.2e-02	5.1e-03	–
value order vs N	2.82	3.08	–
cubic gain at $N = 61$	1.7×	2.7×	–

term to the temporal one. The cubic's advantage shrinks to 1.7–2.7× there, which is what one expects: when the solve itself is wrong, a better reconstruction cannot rescue it. Second, the source is a device for producing an exact non-polynomial solution, not a market friction; the economics of the modified problem are artificial and we do not read anything into them. What the experiment settles is the narrower and more important point: the accuracy comparison in this paper is not an artefact of the benchmark's solution being a polynomial.

C. Value accuracy is not control accuracy

An HJB is solved so that it can be *acted on*, and what is acted on is $u^\star = -R^{-1}B^\top \nabla V$ — a *derivative* of the value function. A grid scheme obtains it in two steps: reconstruct the stored nodal values as a continuous interpolant, then differentiate it. A network obtains it in one, by automatic differentiation, which is exact for whatever function the network represents. We call the first step the *reconstruction*, and its *order* the degree of the local polynomial piece it uses: one for a piecewise-linear (multilinear) reconstruction, three for a cubic one. A reconstruction of order k yields a gradient converging at a rate set by k, and it is that rate — not the accuracy of the solved value function — that governs the control error.

The two errors do not track each other. At every dimension and resolution tested the control error exceeds the value error, and it converges at a stable order of about 1.00: least-squares fits of $\log(\text{error})$ against $\log h$ give 1.00, 0.98, 0.98, 0.98 and 0.98 at $p = 1, \ldots, 5$, with fit R^2 between 0.998 and 1.000. (Every error in this paper is a relative L^2 norm, $\|a - a^\star\|_2 / \|a^\star\|_2$, and every order is fitted to that error rather

than to its square — a convention this paper previously left implicit, which is how the same control error could be read as order 2 in one table and order 1 in another.) The value error converges faster and irregularly — 1.27, 2.19, 1.22, 1.68 and 1.85 over the same five dimensions — and at the finest grid in two of the five dimensions it regresses, so the ratio between the two grows under refinement: at $p = 2$ it runs $1.5 \to 2.2 \to 3.6 \to 5.0 \to 7.0 \to 9.1 \to 13.4$ over $N = 11, \ldots, 61$. The order is unchanged by the signal extension of Section III-A: 1.06 (fit $R^2 = 0.999$, over $N = 11, \ldots, 41$) against 1.05 for the inventory-only benchmark at the same state dimension, so what follows is a property of the differentiation operator and not of the inventory-only model.

The cause is the reconstruction, not the solution. We put the closed-form value function on the grid as exact nodal values, so the discrete solution is error-free by construction and every remaining control error belongs to the step that turns nodal values into a gradient. With the multilinear interpolation and probing difference used above, that step alone reproduces almost all of it: 6.1×10^{-2} against 6.2×10^{-2} at $N = 11$, falling to 1.0×10^{-2} at $N = 61$ at order 0.99, against $6.2 \times 10^{-2} \to 1.1 \times 10^{-2}$ measured on the solved grid. The grid solution is not what limits control accuracy.

What limits it is the order of the reconstruction. Table VI measures the gradient-error convergence rate on functions whose gradients are known exactly, so the rates are properties of the reconstruction rather than of this benchmark. The multilinear reconstruction is first order on every function tested, and taking instead the *exact* gradient of that same interpolant — the best a piecewise-linear reconstruction can do — is also first order, so consistency with the interpolant buys nothing. A cubic reconstruction, being C^2, has a smooth rather than piecewise-constant gradient; with the table's probing step it converges at 2.2 to 2.8 on the non-polynomial test functions, and with a probing step far below the mesh — the last two rows, which measure the reconstruction rather than the probe — the same cubic reaches 3.4 to 3.8. The gap between those two numbers is worth stating plainly: a finite-difference probe whose step is not scaled with the mesh silently caps the order a high-order reconstruction can deliver, so the practical recommendation is not only to raise the order but to differentiate consistently with it. Because this benchmark's value function *is* quadratic and a cubic reconstruction reproduces quadratics exactly, the last column applies the same comparison to the Merton problem, whose closed form is a power function and whose HJB is non-linear in the derivatives; there the cubic order is 2.6 against 1.0. The result is therefore not an artefact of the linear–quadratic structure.

This is why value and control accuracy can be chosen almost independently. Figure 2 is that sentence as a measurement: control error against mesh spacing for the three reconstructions, on the relative L^2 scale used throughout, with the order visible as a slope and the cubic's ceiling visible as the point where the curve stops improving. On the benchmark's own solved grid at $p = 2$, $N = 61$, the multilinear extraction gives a control error of 1.06×10^{-2} against a value error of 7.8×10^{-4},

TABLE VI

THE ORDER OF THE RECONSTRUCTION GOVERNS CONTROL ACCURACY, NOT THE ACCURACY OF THE SOLUTION. EACH ROW TURNS NODAL VALUES INTO A GRADIENT A DIFFERENT WAY: A PIECEWISE-LINEAR (ORDER-ONE) RECONSTRUCTION WHOSE GRADIENT IS PROBED BY FINITE DIFFERENCES, THAT SAME RECONSTRUCTION DIFFERENTIATED EXACTLY, AND A CUBIC (ORDER-THREE) RECONSTRUCTION PROBED EXACTLY AS THE FIRST. THE COLUMNS REPORT THE CONVERGENCE RATE OF THE *gradient* — THE CONTROL — MEASURED ON FUNCTIONS WHOSE GRADIENTS ARE KNOWN EXACTLY, SO THE RATES ARE PROPERTIES OF THE RECONSTRUCTION RATHER THAN OF THE BENCHMARK; THE FIRST FOUR COLUMNS ARE THIS PAPER'S OWN PROBLEM, AVERAGED OVER $p = 2, 3$ AND 5. ORDERS AGREED ACROSS DIMENSIONS TO WITHIN 1.0. EVERY ORDER IS FITTED TO THE RELATIVE L^2 ERROR, AND THE PROBE STEP IS STATED BECAUSE IT IS NOT INNOCENT: THE FIRST THREE ROWS USE THE PRACTICAL PROBE $\varepsilon = h/4$, AND THE LAST ROW REPEATS THE SAME THREE MEASUREMENTS WITH ε FAR BELOW THE MESH, WHICH RECOVERS THE RECONSTRUCTION'S OWN ORDER — THE CUBIC REACHES ≈ 3.5 THERE, SO A PROBE THAT IS NOT SCALED WITH THE MESH SILENTLY CAPS THE ORDER A HIGH-ORDER RECONSTRUCTION CAN DELIVER. THE LAST COLUMN IS THE MERTON VALUE FUNCTION, A POWER FUNCTION FROM A GENUINELY NON-LINEAR CONTROL PROBLEM — NON-POLYNOMIAL, SO THE CUBIC RECONSTRUCTION IS NOT EXACT THERE. DIFFERENTIATING THE PIECEWISE-LINEAR RECONSTRUCTION EXACTLY IS NO BETTER THAN PROBING IT, SO CONSISTENCY WITH THE INTERPOLANT BUYS NOTHING. THE QUADRATIC COLUMN IS THIS PAPER'S OWN CASE AND FLATTERS THE CUBIC METHOD, WHICH REPRODUCES QUADRATICS EXACTLY; EVERY OTHER COLUMN IS THE ONE TO QUOTE.

	gradient-error convergence rate				
reconstruction	quad.	quartic	exp.	trig.	Merton
multilinear + probe	1.1	1.2	1.1	1.1	1.0
multilinear, slope	1.0	1.0	1.0	1.0	1.0
cubic + probe	exact	2.4	2.2	2.8	2.6
multilinear, $\varepsilon \ll h$	1.0	1.0	1.0	1.0	–
cubic, $\varepsilon \ll h$	–	3.4	3.4	3.8	–

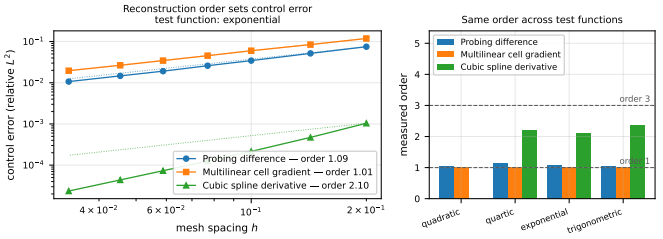


Fig. 2. Control error against mesh spacing, on the relative L^2 scale used throughout. Left, the three reconstructions on one non-polynomial test function, with a slope-1 reference (dotted); the measured orders are fitted to the same data the curves show. Right, the same measurement across four test functions, so the order can be seen to be a property of the reconstruction rather than of the example. The cubic reconstruction is second order with the probing step used here — third order when that step is scaled with the mesh, as the last two rows of Table VI show; the quadratic case is omitted for it because a cubic spline reproduces quadratics exactly, leaving only rounding error to fit.

a ratio of 13.6. The cubic extraction gives 1.3×10^{-3} on the same grid, 8.2 times smaller. There the control error is set by the solver rather than the reconstruction, so the ratio falls to 1.7. The decoupling is largely an artefact of the gradient operator chosen, not a property of grid methods.

The order is a property of the reconstruction, and two points are not enough to show it. The comparison above rests on multilinear against cubic, and two reconstructions can

TABLE VII

THE GRADIENT-ERROR CONVERGENCE RATE AGAINST THE RECONSTRUCTION'S POLYNOMIAL DEGREE, ON A JITTERED MESH. EACH CELL IS A SEPARATE RECONSTRUCTION OF THE SAME NODAL VALUES, AND EVERY DEGREE IS DIFFERENTIATED BY THE SAME RULE – CENTRAL DIFFERENCES ON ITS OWN INTERPOLANT – SO THE ONLY THING VARYING ACROSS A ROW IS THE RECONSTRUCTION. THE RATE RISES WITH THE DEGREE AND THE ERROR AT FIXED MESH FALLS BY FOUR TO FIVE ORDERS OF MAGNITUDE, WHICH MAKES THE PAPER'S RECOMMENDATION ACTIONABLE BEYOND THE TWO RECONSTRUCTIONS IT WAS ORIGINALLY TESTED ON.

degree	exp.	trig.	rational
1	1.05	1.03	1.10
3	3.38	3.53	3.29
5	5.08	6.00	6.34

be ordered by accident. Table VII generalises it: the same nodal values are reconstructed at four polynomial degrees, on three test functions, on a *jittered* mesh, and – the part that makes the comparison mean anything – every degree is differentiated by an identical rule, central differences on its own interpolant, so nothing but the reconstruction varies across a row. The measured gradient convergence rate rises monotonically with the degree, from 1.03–1.10 at degree one to 3.29–3.53 at degree three and 5.08–6.34 at degree five, and at a fixed mesh the error falls by four to five orders of magnitude. These are the reconstruction's own rates: the probe step here is four orders of magnitude below the mesh, which is what makes them comparable with each other. The claim therefore holds along the whole sequence rather than between two chosen representatives, which is what makes the recommendation actionable: a practitioner who wants control accuracy does not choose a better solver family, they raise the order of the operator that turns their stored values into a gradient.

One caution belongs with the degree-five row. On the exponential it first measured 2.89 rather than the 5–6 the other functions give, and the reason is not the reconstruction: at that degree the error reaches the floating-point floor around 10^{-10} and plateaus, so the last few mesh sizes measure rounding rather than convergence and a fit across them reports an order that is far too low. The reported figure excludes the plateau. The same trap produced a spurious order of -2.1 in the earlier comparison, where a cubic reconstruction is exact on a quadratic and leaves nothing but rounding to fit.

Refining in time cannot substitute for any of this. Holding the mesh fixed and varying only the number of stored snapshots separates the two errors cleanly. The value error falls by up to two orders of magnitude ($\times 4.8$, $\times 12.8$, $\times 20.7$ at $p = 2$ for $N = 21, 35, 45$; $\times 67.5$, $\times 131.4$, $\times 65.2$ at $p = 3$), while the control error falls by only $\times 2.0$, $\times 3.2$ and $\times 4.1$ at $p = 2$ and then stops improving; at $p = 5$ it does not improve at all. The plateau is not arbitrary: at $p = 2$ it scales as h to within measurement error.

The resolution an application needs is therefore fixed by the control target *and* the order of the operator used to differentiate the solution — and the second of those is free at the grid level. That is not an argument that control error is harmless: the envelope theorem makes a control error δu perturb the value only by $O(\|\delta u\|^2)$, while the inventory path is a time integral of the control and inherits its error to *first* order. A solver can therefore be accurate enough for valuation and still execute the wrong inventory.

Measured on the executed path, that is what happens. The argument above is an argument; it can be turned into a measurement by closing the loop. Integrating $dx = (Ax + Bu)\,dt$ from a liquidation start under the control each reconstruction actually produces, and comparing against the exact closed loop, separates the consequences cleanly at $N = 11$ (signalled benchmark): the multilinear extraction gives a maximum position error of 1.6×10^{-2} of the initial position and an objective loss of 1.35% of $|J^\star|$, while the cubic extraction on the same nodes gives 4.2×10^{-3} and 3.9×10^{-4}. The ratio between the two quantities grows with refinement — 10.7, 31.3, 55.9, 68.4 over $N = 11, \dots, 31$ — and the fitted orders are 1.7–2.0 for the position error and 2.1 for the objective loss. Proposition IV accounts for both: (9) makes the loss the integral of the squared control deviation, hence second order in a first-order control error, which is what the 2.1 measures; and the position error is bounded by, not equal to, the first-order inheritance $\|x_t - x_t^\star\| \le T\|B\| \sup_t \|\delta u\|$, so an exponent above one is inside the bound rather than in conflict with it. In other words an improvement that a value benchmark cannot see, a $35\times$ reduction in the objective loss, is accompanied by a $4\times$ improvement in the position actually held, and it is the position that a risk limit or an inventory constraint acts on. One limit is worth stating: the controller is stored as snapshots and blended in time, so its time sampling contributes about 10^{-3} to the position error — comparable to the finest-mesh figures — and the fine-mesh rows of that comparison are therefore time-limited rather than reconstruction-limited.

Where the error lives. Accuracy is scored on points drawn uniformly from the state box, and a point within one cell of the box's edge lies where the solver's stencils are one-sided and the reconstruction has information on one side only. That band is 13% of the domain at $p = 1$ and 33% at $p = 3$ for $N = 21$ — 41% at $p = 5$ by the same formula — so a single domain-wide average could be dominated by it without anyone noticing. It is not: the control-error density in the band is 0.99–1.00 times the interior's at $p = 1, 2, 3$, and the band contributes exactly its share of the domain to the total squared error. Three things belong with that check. The solvers pad the domain before building the grid, so the grid's own outermost cells are never queried at all; the statement is about the problem domain, not the grid edge. The measurement is easy to get backwards — normalising the error inside each region makes the boundary look *more* accurate, because $|V|$ is largest there, which is what our first version of it reported. And one asymmetry does survive: the cubic reconstruction's advantage is smaller in the band (14–$19\times$) than in the interior ($26\times$), which is what one expects when there is no node on the far side.

What the improvement is worth in money. The comparison above is in relative L^2 units, which is not what an execution

TABLE VIII

THE RECONSTRUCTION ORDER IN THE CURRENCY IT TRADES IN. EACH INSTRUMENT IS RUN THROUGH THE EXECUTED CLOSED LOOP WITH THE PAPER'S REAL-MARKET CALIBRATION (η = HALF THE ROLL EFFECTIVE SPREAD, σ FROM MINUTE BARS), WITH THE EXACT VALUE FUNCTION PLACED ON THE GRID SO THE SOLVE CONTRIBUTES NOTHING AND ONLY THE RECONSTRUCTION DIFFERS. *Excess over exact* IS THE COST OF AN ARM MEASURED AGAINST THE EXACT CONTROL, IN BASIS POINTS OF NOTIONAL; *multilinear vs cubic* IS THE PAIRWISE DIFFERENCE, WHICH IS THE WELL-POSED STATEMENT BECAUSE BOTH ARMS SHARE THE SNAPSHOTS AND THE TIME INTERPOLATION. THE CUBIC ARM SITS AT THE NUMERICAL FLOOR (≤ 0.02 BP), SO RAISING THE RECONSTRUCTION ORDER IS WORTH THE DIFFERENCE COLUMN — AND IT IS WORTH MOST ON THE MOST LIQUID INSTRUMENT, BECAUSE A TIGHTER SPREAD BUYS A LARGER OPTIMAL CONTROL AND THEREFORE A LARGER RECONSTRUCTION ERROR. FULL HORIZON: TRUNCATING THE INTEGRAL AT $T - \delta$ REMOVES THE TERMINAL PENALTY THAT PAYS FOR NOT LIQUIDATING, WHICH MAKES THE SLOWER POLICY LOOK CHEAP (THE PAIRWISE FIGURE FALLS TO -0.21 BP AT $\delta = 0.1$ ON SPY); THE SENSITIVITY IS IN THE REPRODUCTION.

instrument	η (bp)	excess over exact: ml	ml vs cubic
SPY	1.28	8.42	8.40
AMZN	1.46	7.45	7.43
MSFT	1.58	6.91	6.89
QQQ	1.60	6.82	6.81
GOOGL	1.97	5.60	5.59
OXY	2.24	4.97	4.96
NVDA	2.68	4.21	4.20
AAPL	2.73	4.16	4.14
TSLA	4.41	2.69	2.67
mean 5.68 bp of notional			

desk books. Table VIII closes the loop on the paper's own real-market calibration. For each of the nine instruments the exact value function is placed on the grid, so the solve contributes no error and the reconstruction is the only thing that differs; each reconstruction then drives the deterministic closed loop; and the cost is the benchmark's own objective. Its units are money once σ and η are relative — $\frac{1}{2}\int \eta u^\top u\, dt$ is the impact paid as a fraction of notional, and 10^4 times it is basis points. Raising the reconstruction from multilinear to cubic is worth 2.7 to 8.4 basis points of notional, a mean of 5.7 bp — about \$5.7k on a \$10M order — while the cubic arm itself sits within 0.018 bp of the exact control — the pairwise difference is therefore a statement about the multilinear arm, which is what makes it quotable. The ordering is the same one Section VI-H found on the cost side and for the same reason: the effect is largest on the *most* liquid instrument, because a tighter spread buys a larger optimal control and therefore a larger reconstruction error $\delta u \sim P\, h/\eta$. Two measurement points belong with the number. It is the pairwise difference over the full horizon, not the excess over the exact control truncated at $T-\delta$. Truncation removes the terminal penalty that pays for not liquidating, so the slower arm looks cheap and the difference falls to -0.21 bp at $\delta = 0.1$. That is the same terminal contamination the paper documents for accuracy metrics, in its cost form, with the opposite remedy: keep the horizon here, drop the terminal sample there. And the figure is insensitive to the one coefficient that cannot be measured: across $\lambda \in [10^{-6}, 10^{-2}]$ it moves by less than 0.002 bp.

D. Dimension and parameterisation

The sweep above stops at $p = 5$ because grid methods stop being runnable past that. The analytic reference does not: the Riccati equation is a fixed number of ODEs whatever the state dimension, so accuracy can still be *measured* where no classical baseline can be run, and we extended the range to $p = 20$. Doing so exposed a flaw in the sweep above. With the uniform defaults the value function is *separable*, $V(x,t) = \sum_i v(x_i, t)$, so solving it at $p = 20$ is barely harder than at $p = 1$: the network applies one learned univariate function and sums. That is not the case the curse-of-dimensionality argument is about.

The signal extension of Section III-A removes the symmetry by construction rather than by assumption: with *perfectly uniform* parameters the off-diagonal block of P reaches 0.205 of the diagonal, against exactly zero here. The contrast at *matched state dimension*, with the same architecture, budget and two seeds, is stark:

d	separable	with a signal
2	$+0.9994 \pm 0.0002$	$+0.9993 \pm 0.0002$
4	-44.5 ± 41.2	$+0.9988 \pm 0.0001$
6	-646.6 ± 0.3	$+0.9968 \pm 0.0003$

At $d = 4$ and $d = 6$ the separable problem is not merely less accurate but unsolved — a negative R^2 means the learned value function is worse than predicting the mean — while the harder, genuinely multi-dimensional problem is solved to three decimals. Randomising each asset's risk and impact coefficients reproduces the same contrast on the inventory-only problem (-688 against 0.981 at $p = 8$), which is how we first saw it; the signal version is the one to quote, because it needs neither a random draw nor the claim that a particular draw is representative. The failure is a property of the parameterisation, not of the dimension.

On the non-separable problem the PINN does not collapse; it degrades with dimension, giving $R^2 = 0.99999$, 0.9994, 0.9992, 0.977, 0.981, 0.927 and 0.757 at $p = 1$, 2, 3, 5, 8, 12 and 20. A grid baseline remains available and more accurate through $p = 5$ (0.9999 down to 0.9993) but is not runnable at $p \geq 8$, where N^d alone is 2×10^7 points before the CFL-limited time step is counted. Beyond that the mesh-free solver is the only method that runs at all — at accuracy that declines to 0.76 by $p = 20$ and that the user cannot bound, because nothing is left to compare against except the analytic solution we happen to have here.

The practical guidance is therefore narrower than "PINNs fail at high dimension". What fails is a combination of architecture, parameterisation and budget, and the same architecture succeeds at the same dimension once the problem is made generic — so a PINN validated only on a symmetric toy problem has learned nothing about either outcome.

One objection is under-training, and it deserves a direct answer. At $p = 4$ the fixed budget of 6000 epochs and 2048 collocation points yields $R^2 = -44.5$ in 177 s; 20000 epochs at the same batch yields $R^2 = +0.9972$ in 630 s. At $p = 3$ the same increase moves R^2 from 0.9981 in 172 s to 0.9997

in 541 s. Budget is therefore one lever. It is not the only one, and saying so was an over-reach in an earlier draft of this paragraph: Section VI-F shows that at the *unchanged* budget a deeper network of the same family also recovers this dimension, so the failure is a statement about the combination in front of it, not about budget alone. What does not change under either route is the cost: the rescued $p = 4$ run took 630 s against 31.7 s for accuracy-matched finite differences — about $20\times$ more expensive, for lower accuracy ($R^2 = 0.9972$ versus 0.99991). Since the neural solver's cost grows only slowly in p while the grid scheme's grows geometrically, the gap must close eventually, but over $p = 1$ to 5 it does not, and at each dimension where both methods succeed the grid scheme is faster by a factor of 5 to 50 — at the coefficients benchmarked here, a scope Section VI-H measures.

E. The mesh-free region is not the neural solver's alone

Everything above compares a learned solver with a mesh-based one. That leaves out the classical family that also needs no mesh — simulation with backward regression, the least-squares Monte Carlo line [34], [35] and its stochastic-control variants [36], [37] — and leaving it out is not neutral: it is the family a referee would name, and it decides whether the boundary drawn above is between *learned* and *classical* or merely between *what we ran* and *what we did not*. So we run it.

The scheme is policy iteration with backward regression, on the same problem and the same reference as the high-dimensional sweep of Section VI-D. Sample states across the domain; simulate paths under the current policy; regress the realised future reward at each time slice on the quadratic features $1, x_i, x_i x_j, \frac{1}{2}x_i^2$; improve the policy to $u = R^{-1}B^\top \nabla V$ from the fitted quadratic; repeat. Three implementation facts belong in the record, because each of them was needed to make the method work at all and none of them is free. Paths must start *across the domain*: started from x_0 alone the cloud has radius $\sigma\sqrt{T} = 0.01$ around one corner of the box, the fitted Hessian is meaningless, and the extracted control diverges — to 10^{33} in two iterations when we did that. The greedy update must be *damped*: with a gain of $1/\eta = 100$, a Hessian error of 10^{-3} is a feedback-gain error of 0.1, and replacing the policy outright is unstable here (we report $\alpha = 0.5$). And the regression must include the exact terminal condition, which is part of the problem rather than part of the answer.

Table IX is the result, and it changes a conclusion. Regression Monte Carlo is more accurate than the neural solver at every dimension where both were measured — at $d = 8$ by 0.9988 against 0.9813, at $d = 12$ by 0.9989 against 0.9269, and at $d = 20$ by 0.9771 against 0.7570 — and it does so in 2.7, 5.1 and 12.5 seconds on a CPU where the neural runs took 196, 270 and 414 seconds on a GPU. Its control is more accurate still. So "beyond the grid's reach the neural solver is what remains" is false as stated; what remains is mesh-free methods, of which one is classical, cheaper and — here — better.

TABLE IX

BEYOND THE GRID'S LIMIT, ON THE NON-SEPARABLE BENCHMARK (PER-ASSET HETEROGENEOUS RISK AND IMPACT, RANDOM OFF-DIAGONAL RISK). THE MESH-FREE SIDE IS NOT ONLY NEURAL: REGRESSION MONTE CARLO — A CLASSICAL METHOD THAT NEEDS NO GRID — RUNS AT EVERY DIMENSION HERE, AND IT IS MORE ACCURATE THAN THE NEURAL SOLVER AT EVERY DIMENSION WHERE BOTH WERE MEASURED, IN SECONDS ON A CPU WHERE THE NEURAL RUNS TAKE MINUTES ON A GPU. m IS THE NUMBER OF QUADRATIC BASIS FUNCTIONS THE REGRESSION FITS AT EVERY TIME SLICE, $1 + d + d(d+1)/2$: THE COST OF THIS FAMILY GROWS IN THE BASIS, NOT IN THE STATE DIMENSION. THAT BASIS IS ALSO THE QUALIFICATION — IT IS THE EXACT STRUCTURE OF THIS BENCHMARK'S VALUE FUNCTION, WHICH THE NEURAL SOLVER IS NOT GIVEN, SO THE RESULT IS A STATEMENT ABOUT THIS PROBLEM CLASS RATHER THAN ABOUT HIGH-DIMENSIONAL HJB EQUATIONS IN GENERAL. PINN ENTRIES ARE MEANS OVER ITS SEEDS AT A FIXED 8000-EPOCH BUDGET; THE GRID IS RUNNABLE ONLY TO $p = 5$.

d	PINN R^2	RMC R^2	RMC m	RMC (s)	grid R^2
1	$+1.0000$	$+1.0000$	3	0	$+0.99989$
2	$+0.9994$	$+0.9998$	6	0	$+0.99992$
3	$+0.9992$	$+0.9998$	10	1	$+0.99993$
5	$+0.9771$	$+0.9995$	21	2	$+0.99929$
8	$+0.9813$	$+0.9988$	45	3	n/a†
12	$+0.9269$	$+0.9989$	91	5	n/a†
20	$+0.7570$	$+0.9771$	231	13	n/a†

†not runnable: N^p grid points at the resolution the accuracy target needs.

Two qualifications keep that from being over-read, and the first is the important one. The regression uses a quadratic basis, which is the exact structure of this benchmark's value function; the neural solver is given no such structure. That is a real advantage and it is the reason the comparison is a statement about *this benchmark class* rather than about high-dimensional HJB equations in general. Where the value function is not close to quadratic the basis has to be enlarged. The cost of this family grows as $1 + d + d(d+1)/2$ basis functions (231 at $d = 20$) rather than in the state dimension, and Section VI-B shows what happens when the solution stops being quadratic. The honest reading is that the classical mesh-free method wins where the practitioner has structural knowledge, and the neural solver is the fallback where they do not.

That qualification is measured rather than asserted. Running the same regression, with the same quadratic basis, on the manufactured family of Section VI-B — whose value function is progressively less quadratic — gives a realised accuracy of $R^2 = 0.9998$ with the basis exactly right ($A_0 = 0$), 0.9995 at $A_0 = 10^{-4}$, 0.956 at $A_0 = 10^{-3}$, and -1.19 to -1.20 at $A_0 = 5 \times 10^{-3}$, which is the amplitude of the non-polynomial term used in Section VI-B. At $A_0 = 2 \times 10^{-2}$ the run diverges: the fitted policy drives the state out of the region the basis can describe, and the regression becomes ill-posed. The figure at the failing amplitude is quoted as a range on purpose: within one run the seeded repeats agree to 0.01, but across independent runs of the same script it moves by up to 0.07, which is part of the finding rather than an inconvenience — a method in a failing regime does not fail reproducibly. So the classical advantage is worth exactly what structural knowledge of the value function is worth, and it fails outright — not merely degrades — when that knowledge is wrong by an amount this benchmark can produce. The practitioner's

TABLE X

ARCHITECTURE SENSITIVITY AT $p = 4$, THE DIMENSION WHERE THE FIXED BUDGET COLLAPSES. EVERY ROW IS A FULL SOLVE SCORED AGAINST THE ANALYTIC REFERENCE; THE FIRST ROW IS THE ACCURACY-MATCHED GRID SCHEME AT THE SAME DIMENSION, FOR SCALE. TWO SEPARATE QUESTIONS ARE ANSWERED HERE. DOES MORE CAPACITY HELP AT A FIXED BUDGET – YES, DECISIVELY: AT 6,000 EPOCHS THE PAPER'S 4×64 SCORES $R^2 = -3.28$ WHILE AN 8×64 MLP, USING $2.3\times$ THE PARAMETERS, SCORES $+0.9953$. AND IS CAPACITY THE ONLY LEVER – NO: RAISING THE BUDGET OF THE ORIGINAL ARCHITECTURE TO 20,000 EPOCHS REACHES $+0.9972$ FOR LESS WALL TIME THAN THE LARGEST NETWORK HERE. BOTH ROUTES SUCCEED, AND BOTH REMAIN SLOWER AND LESS ACCURATE THAN THE GRID SCHEME.

net	ep.	params	time (s)	value R^2	control R^2
grid ($N = 21$)	—	—	20	$+0.99991$	$+0.99603$
2x64 shallow	6k	4,609	61	-337.02583	-1.0963
4x64 (paper)	6k	12,929	85	-3.27683	$+0.7316$
8x64 deep	6k	29,569	128	$+0.99530$	$+0.9969$
4x128 wide	6k	50,433	79	$+0.93647$	$+0.9869$
6x256 both	6k	330,753	100	$+0.99672$	$+0.9977$
2x64 shallow	20k	4,609	208	$+0.98600$	$+0.9874$
4x64 (paper)	20k	12,929	290	$+0.99719$	$+0.9976$
8x64 deep	20k	29,569	450	$+0.99956$	$+0.9993$
4x128 wide	20k	50,433	288	$+0.99974$	$+0.9992$
6x256 both	20k	330,753	351	$+0.99757$	$+0.9976$

decision is therefore not "classical or neural" but whether the structural assumption can be checked, and the same repository runs the check. Second, the neural numbers here come from a fixed 8000-epoch budget applied at every dimension rather than tuned per dimension, and this paper's own ablation shows that budget moves the boundary; the comparison is therefore at stated budgets, not at matched accuracy. Both point the same way for a practitioner: the neural solver's niche is narrower than the framing it is usually given, and the measurement that decides it is cheap to run.

F. How much does the architecture choice matter?

Table X answers the objection this paper would otherwise have to concede, that a single architecture cannot separate a property of the solver family from an artefact of the network. The sweep is run at $p = 4$, the dimension where the fixed budget collapses, because that is where an architecture effect has room to show; a dimension where the solver already succeeds would measure nothing. Each row is a full solve, and the grid scheme is included only for scale.

Three things follow, and the third is the one that matters.

Capacity does move the boundary. At the paper's own 6000-epoch budget the 4×64 MLP scores $R^2 = -3.28$ while an 8×64 MLP scores $+0.9953$ for $2.3\times$ the parameters and $1.5\times$ the wall time. A collapse at a fixed budget is therefore not evidence that the family is capacity-limited at that dimension; it is evidence that this architecture is.

Depth is the lever, not width. At the same budget the 8×64 network (29,569 parameters) reaches $+0.9953$ while the wider 4×128 (50,433 parameters) reaches only $+0.9365$. More parameters bought a worse result, so the useful move is deeper composition of the scaled inputs rather than a bigger final layer.

TABLE XI

DEEP BSDE, THE STANDARD HIGH-DIMENSIONAL METHOD IN MATHEMATICAL FINANCE, ON THE SAME PROBLEM AND AT THE SAME DIMENSIONS AS TABLE II. TWO COLUMNS ARE DELIBERATELY NOT COMPARABLE WITH THAT TABLE AND ARE LABELLED SO. FIRST, THE ERROR IS MEASURED AT THE SINGLE INITIAL STATE THE SOLVER ANSWERS FOR (x_0, ONE UNIT OF EACH ASSET), WHEREAS THE GRID AND PINN COLUMNS OF TABLE II ARE DOMAIN-WIDE: A POINT-VALUED SOLVER IS NOT BEING CREDITED WITH A FUNCTION-VALUED ONE'S ACCURACY. SECOND, THE VALUE IS QUOTED *at its best point during training*, NOT AT THE END. IN EVERY RUN THE TERMINAL RESIDUAL KEPT FALLING WHILE THE VALUE ERROR BOTTOMED OUT AND THEN DEGRADED, SO THE ENDPOINT IS A FACTOR OF 14 TO 686 WORSE THAN THE NUMBER HERE, AND A READER WHO TRAINS TO A RESIDUAL PLATEAU WILL NOT REPRODUCE THIS COLUMN. TIME IS TRAINING WALL TIME TO THAT STOPPING POINT, AT THE PER-EPOCH RATE MEASURED FOR EACH DIMENSION IN A SINGLE CONTROLLED RUN; THAT RATE IS NEARLY FLAT IN p (0.11–0.15 S PER EPOCH), SO THE COST IS CARRIED BY THE NUMBER OF EPOCHS NEEDED TO CONVERGE, NOT BY THE COST OF AN EPOCH. ONE TRAINING RUN ANSWERS ONE INITIAL STATE; IT DOES NOT RETURN V OVER THE DOMAIN.

p	n_{steps}	train (s)	rel. err at x_0	domain var	resid. at stop
1	200	309	0.053	4.22e-06	1.76e-03
4	50	192	0.033	1.62e-05	5.21e+00
8	50	759	0.003	2.76e-05	7.17e-01

Both routes work, and neither reaches the grid. Raising the budget of the *original* architecture to 20000 epochs reaches $+0.9972$ in 290 s. The 331k-parameter network is more accurate at $+0.9976$ but costs 351 s to get there, so on accuracy per second the small network given more steps is the better trade — which is the direction a practitioner should take from this table. At 20000 epochs four of the five architectures exceed $+0.9995$ and the architecture effect is small: what looked like a capacity story at 6000 epochs is largely a budget story, exposed by giving capacity somewhere to go. Across all eleven configurations the best neural result is $+0.99974$ in 288 s, against the accuracy-matched grid at $+0.99991$ in 20 s. The separation this paper reports therefore does not depend on which network is chosen, which is the claim the single-architecture design could not previously support.

The sweep is over MLPs at a fixed optimiser and a fixed collocation scheme. It does not exclude operator architectures, spectral features or curriculum on the sampling, and the honest reading of "no architecture search" is that this bounds the effect of capacity within one family rather than the effect of architecture as such.

The omission that most needs answering is deep BSDE, which is the standard high-dimensional method in mathematical finance and which we therefore ran rather than merely cited. Table XI gives it on the same problem at $p = 1$, 4 and 8, and the comparison is deliberately narrow: deep BSDE is point-valued, so its error is measured at the single initial state it answers for, not over the domain as the grid and PINN columns of Table II are. Two things follow.

First, where the grid scheme can run, deep BSDE does not reach its accuracy. At $p = 1$ and $p = 4$ its best value error is

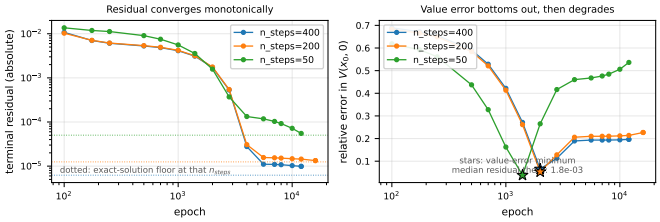


Fig. 3. Deep BSDE: the terminal residual converges, the value does not. Left, the residual falls monotonically at every resolution, approaching the discretisation floor of the scheme it is solving (dotted). Right, the same runs' relative error in $V(x_0, 0)$ bottoms out and then *grows* (stars mark each minimum). Training to a residual plateau therefore overshoots the best value the run passes through, by a factor of 14 to 686 depending on dimension — which is why Table XI quotes the value at its minimum and not at the end.

5.3% and 3.3%, against value R^2 of 0.99995 and 0.99992 for accuracy-matched finite differences, and its terminal residual at that point is still $71\times$ and $2.6 \times 10^4\times$ the discretisation floor of the scheme it is solving. So the conclusion of Table II is not an artefact of having chosen the PINN as the neural representative: the other standard neural family, measured rather than assumed, loses to the classical scheme on the same axis in the regime where the classical scheme is available.

Second, and in the other direction, deep BSDE is the strongest of the learned solvers where the grid is not runnable. At $p = 8$ it reaches 0.35% relative error on $V(x_0, 0)$, while the PINN reaches $R^2 = 0.977$ and the grid scheme is 2×10^7 points before its CFL-limited time step is counted. The cost is not the per-epoch rate — that is nearly flat in p, 0.11 to 0.15 s — but the number of epochs needed to reach the stopping point, which rises from 2000 at $p = 4$ to 6000 at $p = 8$.

One caution belongs with any deep-BSDE number, and Figure 3 is the reason. In every run the residual kept falling while the value error degraded, reaching 1268% by the end of the $p = 4$ run. The stopping point is consequently part of the reported configuration. It does not transfer between dimensions: the absolute residual threshold that is optimal at $p = 1$ is three orders of magnitude too late at $p = 4$. A scale-free version still moves by a factor of 31 across the dimensions we measured, so the stopping point has to be found by monitoring the value rather than by a rule.

And the family's modern member. A referee is entitled to answer the above with "then compare against a strong variant". Deep splitting [17] is that variant: instead of one time-global network it fits one small network per time step by regression on simulated paths, and we implement it, validate it and run it on the same benchmark. The scheme itself is sound here — running the same recursion with the per-step regression solved exactly in the quadratic basis, which is the solution's own functional form, gives value R^2 of 0.9987, 0.9960 and 0.9939 at $d = 1, 4, 8$ — but the neural realisation does not reach it: 0.16, -76 and -286 at the same step count, with the median per-step regression loss degrading from 0.05 to 1.0 over the same range. The discrepancy is therefore the learner's, not the scheme's, and the mechanism is the one this paper is about. The recursion feeds the *gradient* forward: $z = \Sigma^\top \nabla u$ enters $f = -\frac{1}{2} x^\top Q x + \frac{1}{2} z^\top M z$ with $M = \Sigma^{-1} B R^{-1} B^\top \Sigma^{-\top}$ of order 10^6, so a per-step regression is asked for gradient accuracy, while a mean-squared-error loss constrains the function and not its derivative. This is the value/control decoupling of Section VI-C reappearing inside a neural solver, with the same practical reading: a splitting implementation validated on its value residual can be worthless in the value it reports while its control looks reasonable — R^2 of 0.86 on the control against 0.16 on the value at $d = 1$. Deep splitting is not a weak method; on this benchmark it is a method whose per-step regression has to be accurate in a derivative, and we report both arms so that the reader can see which one the limitation belongs to.

That the limitation is neither the training budget nor the network's capacity is measured rather than assumed. At a fixed 50 steps, moving from 2×32 and 300 epochs per step to 4×64 and 2000 — $6.7\times$ the gradient steps — improves the median per-step fit by $179\times$ (to 2.7×10^{-4}), and moves the value R^2 from $+0.16$ to -16.5: a better fit to each step's target produces a worse value function, while the control R^2 stays between 0.86 and 0.89 throughout. The corresponding check on the mechanism is direct: at the trained networks the *relative* error of the recovered gradient exceeds that of the value by 1.6 to $4.3\times$ at the steps we measured (with one exception near $t = 0$), which is what a loss constraining the function rather than its derivative predicts, and it is the gradient that the recursion feeds forward.

G. The classical probabilistic method, where it applies

The last family to place is the one with a complexity theory behind it, and Table XII gives both what it achieves and the condition under which it achieves it. Multilevel Picard [23], [52] approximates a high-dimensional semilinear PDE at a point by realising the Picard iteration in a full-history multilevel Monte Carlo scheme, and it is the classical method the deep-learning literature is usually measured against on high-dimensional problems. We implement it in the published form, with one deviation stated rather than hidden: our nonlinearity consumes the *gradient*, so ∇U_ℓ is obtained by central differences with step ε — the practical route for gradient-dependent nonlinearities [53] — rather than by the Beta-time-sampled variant that reference introduces for exactly this case. Two consequences follow, and the second is the finding.

First, where the method's hypothesis holds it is the strongest thing in this paper, and not narrowly. MLP realises the Picard iteration, whose Lipschitz constant here is $|\nabla V|/\eta$; at $\eta = 1$ (a wider spread, i.e. weaker control authority) that is 0.10 and the iteration converges. It then reaches 0.26% relative error on $V(x_0, 0)$ at $d = 20$ in 2.3 seconds on a CPU, against $R^2 = 0.757$ for the PINN on the same problem, 0.977 for regression Monte Carlo (in 12.5 s, and with a hand-chosen basis), and no grid at all. The implementation is checked where the hypothesis holds — the relative error falls 6.7×10^{-3} to 1.5×10^{-3} from level 2 to level 4 — so the divergence below is the method's condition and not a defect in our code.

Second, at the paper's own calibration it does not converge, and the boundary is measurable rather than mysterious. The Pi-

TABLE XII

MULTILEVEL PICARD, MEASURED RATHER THAN CITED. *Upper*: THE METHOD REALISES THE PICARD ITERATION, SO IT NEEDS THAT ITERATION TO BE A CONTRACTION, WITH LIPSCHITZ CONSTANT $|\nabla V|/\eta$. SWEEPING η FROM WEAK CONTROL AUTHORITY TO STRONG LOCATES THE BOUNDARY BETWEEN $|\nabla V|/\eta = 0.56$ (CONVERGED) AND 0.91 (DIVERGED) INSTEAD OF LEAVING IT AS A CAVEAT. *Lower*: THE SAME HJB FAMILY AT $d = 1$ TO 20. AT THE PAPER'S OWN CALIBRATION THE SCHEME DIVERGES AT EVERY DIMENSION — IN A TENTH OF A SECOND, SO THIS IS THE HYPOTHESIS FAILING RATHER THAN A BUDGET RUNNING OUT — WHILE AT $\eta = 1$ IT IS THE CHEAPEST METHOD IN THIS PAPER, 0.26% AT $d = 20$ IN 2.3 S ON A CPU, WHERE NO GRID CAN RUN AND WHERE THE PINN REACHES $R^2 = 0.757$. THE IMPLEMENTATION IS CHECKED WHERE THE HYPOTHESIS HOLDS: AT $\eta = 1$ THE RELATIVE ERROR FALLS $6.7e-03 \to 1.5e-03$ FROM LEVEL 2 TO LEVEL 4.

contraction boundary at $d = 1$			
η	$\lvert\nabla V\rvert/\eta$	level-3 rel. err	verdict
10	0.011	$7.02e-03$	converged
1	0.100	$6.37e-03$	converged
0.3	0.275	$1.06e-02$	converged
0.1	0.557	$2.12e-02$	converged
0.03	0.907	$4.38e+00$	diverged
0.01	1.249	$8.21e+02$	diverged

across dimension, $M = 2$, 3 levels			
d	rel. err at $\eta = 0.01$	rel. err at $\eta = 1$	time at $\eta = 1$ (s)
1	82115%	0.637%	0.00
4	582833%	0.439%	0.05
8	551822%	0.425%	0.20
12	433216%	0.173%	0.58
20	201008%	0.258%	2.25

card map's Lipschitz constant at $\eta = 0.01$ is 1.25: above one, so the iteration is not a contraction and the iterates diverge. Sweeping η locates the transition between $|\nabla V|/\eta = 0.56$ (converged) and 0.91 (diverged). The divergence is not a budget running out — the $d = 20$ run that diverges takes 1.8 s — and it is not the finite-difference gradient, whose step we varied over two orders of magnitude with no effect on the outcome. It is the same stiffness that sets the grid's CFL limit and destabilises the explicit splitting recursion: the maximised Hamiltonian $\frac{1}{2}\nabla V^\top B R^{-1} B^\top \nabla V$, whose coefficient is $1/\eta$, is what has to be handled carefully by anything that touches it, and multilevel Picard touches it through a contraction. This is the third method in this paper to be limited by that one term, and it is the term the optimal-execution literature treats as a modelling constant rather than as a numerical parameter.

H. Market-calibrated scope

Everything above uses coefficients that were chosen, not measured: $\eta = \lambda\sigma^2 = 0.01$ and $Q_T = 0.1$. A benchmark is entitled to do that, but the entitlement expires when its conclusions are read as statements about trading, so this subsection asks what happens when the coefficients come from the data instead. Table XIII answers it for the impact coefficient.

Two of the three are measurable and one is not. The diffusion scale is the relative volatility, which we take from the minute bars. The impact coefficient is not observable, and we identify it with half the Roll effective spread [39] – a round trip pays the spread once, so a unit trade pays about half of it – using the estimator already computed on the causal data and cross-checked against Corwin–Schultz [40]. Risk aversion is a preference. Nothing in a price series determines it, so any single calibrated value would be a choice dressed as a measurement.

TABLE XIII

THE EXPLICIT GRID SCHEME'S TIME-STEP REQUIREMENT UNDER MEASURED LIQUIDITY. η IS HALF THE ROLL EFFECTIVE SPREAD [39] MEASURED ON THE CAUSAL MINUTE DATA, WITH CORWIN–SCHULTZ [40] AS A CROSS-CHECK. THE OPTIMAL CONTROL IS $u^\star = -R^{-1}B^\top Px$ WITH $R = \eta I$, AND THE EXPLICIT TIME STEP IS CFL-LIMITED BY $dt \le 0.25\,dx/\|u^\star\|_\infty$, SO A SMALLER η BUYS A LARGER CONTROL AND A PROPORTIONALLY SMALLER STEP. EVERYTHING ELSE IS HELD AT THE PAPER'S SYNTHETIC VALUES, SO THE COLUMN IS A STATEMENT ABOUT THE INSTRUMENT'S LIQUIDITY ALONE. THE ORDERING IS THE POINT: THE MOST LIQUID INSTRUMENT PAYS THE LARGEST PENALTY.

instrument	half-Roll spread η	$\max\lvert u^\star\rvert$ on grid	time steps	vs. synthetic calibration
synthetic	1.00e-02	12.5	800	1×
SPY	1.28e-04	975	62,427	78×
AMZN	1.46e-04	858	54,889	69×
MSFT	1.58e-04	792	50,713	63×
QQQ	1.60e-04	781	50,016	63×
GOOGL	1.97e-04	634	40,547	51×
OXY	2.24e-04	557	35,667	45×
NVDA	2.69e-04	466	29,795	37×
AAPL	2.73e-04	458	29,342	37×
TSLA	4.41e-04	283	18,124	23×

That last point sets the design rather than excusing it. On this problem the pinned quantity is $k = \sqrt{Q/\eta} = \sigma\sqrt{\lambda}/\sqrt{\eta}$, which fixes how fast the position is liquidated. Measured liquidity with a defensible $\lambda \in [10^{-6}, 10^{-2}]$ puts k between 0.001 and 0.15; our synthetic coefficients sit at $k = 1$. The benchmark therefore liquidates roughly an order of magnitude faster than real liquidity and realistic risk aversion imply. We sweep λ across that range rather than picking from it, so the results below are a statement about robustness and not about a chosen calibration.

Accuracy is unaffected. Across thirteen configurations – SPY, AAPL, NVDA and TSLA, each at three values of λ, at $p = 1$ – the grid scheme scores value R^2 between $+0.99996$ and $+0.99997$ against $+0.99999$ at the synthetic coefficients, with control R^2 unchanged at $+0.9990$. The control-to-value error ratio, the quantity Section VI-C is built on, lands between 7.9 and 16.4 across every configuration, against 16.4 synthetic. Real coefficients move neither the accuracy of the classical scheme nor the size of the control/value gap.

Cost does not survive, and this is the finding. The optimal control is $u^\star = -R^{-1}B^\top Px$ with $R = \eta I$, so a smaller impact coefficient buys a proportionally larger control; the explicit scheme's step is CFL-limited by $dt \le 0.25\,dx/\|u^\star\|_\infty$; and the step count therefore scales as $1/\eta$. Table XIII evaluates that at each instrument's own measured spread, holding everything else at the paper's values. The tightest instrument in the collection, SPY, needs 78× the synthetic run's time steps; the widest, TSLA, needs 23×. The ordering is the opposite of the intuitive one: liquidity is what makes the explicit scheme expensive here, not illiquidity.

So the paper's speed claim has a narrower scope than its accuracy claim, and we state it that way. Where a grid scheme can run it is faster than the mesh-free solver *at the coefficients benchmarked here*; transferring that to a specific instrument requires the step count to be redone at that instrument's spread. Two qualifications keep this from over-reading. It constrains the *explicit* scheme, which is the one our cost tables use; our implicit variant (Section VII) removes the stability restriction at four times the explicit step, but it was measured at the synthetic η only, so nothing here says an implicit scheme is slow at real spreads. And a step count is work, not wall-clock: the grid's per-step cost is the point count, so the ratio of step counts is the ratio of work for a fixed mesh.

One methodological consequence is worth recording because it cost us a wrong result first. The accuracy function samples query times with `linspace`, which includes $t = T$, and the reference control diverges there – the last instant of a liquidation problem wants to trade infinitely fast. At the synthetic η the terminal sample is mild, with mean $|u^\star| = 5.1$ against 0.6 to 1.7 in the interior, and excluding it moves control R^2 by less than 10^{-4}. At SPY's η, smaller by $78\times$, the terminal sample reaches 395 and a solve that is accurate in the interior is scored at value $R^2 = -997$. The estimator is therefore not the difficulty; the sampling is, and any evaluation at real spreads has to drop the terminal sample.

VII. Discussion and Limitations

A. When to use which

Table XIV states the recommendation the comparison supports. Every row is a regime a practitioner can recognise before committing to a method, and every recommendation points at a measurement in this paper rather than at a preference. The neural solver is preferred in only two rows, and in both the reason is that the classical scheme cannot be run or should not be re-run — not that it was beaten on accuracy; where the two can be compared, the classical scheme wins on both axes.

B. One problem or a family

A natural objection is that the comparison should use inference time rather than training time, since a trained network evaluates in microseconds. We measured both, and the objection does not hold here: both methods have an expensive offline solve followed by cheap online queries. Amortised over one trading day of minute decisions the grid scheme's interpolation costs $0.66\,\mu$s at $p = 1$ and $34.3\,\mu$s at $p = 5$, against 3.0–4.9 μs for the PINN on GPU, so the neural solver's inference is *not* uniformly faster. Combining the terms as $T(N) = T_{\text{offline}} + N\,t_{\text{eval}}$, the PINN never breaks even at $p = 1$, and at $p = 5$ it is finally cheaper on *both* terms (226 s against an 865 s solve) only where it is not accurate ($R^2 = -616$): the binding constraint has moved from cost to accuracy.

The regime changes for a *family* of problems. Where the solver must be re-solved for every member, the break-even count with 10^3 training solves is $4{,}365$ instances at $p = 2$ and $1{,}000$ at $p = 5$ — essentially the size of the training set, and *falling* as dimension rises — so an *operator* amortises after $1{,}000$ to $4{,}365$ parameter settings, which is exactly the regime a calibration study occupies. A fixed-parameter PINN does not, and most operator architectures would need labelled solutions from the solver they aim to replace, though the physics-informed family [54] trains on the residual and needs none.

Operator learning, and why it is a different axis. The field has moved from fixed architectures — the spectral

TABLE XIV
WHICH SOLVER TO USE. EACH ROW IS A REGIME A PRACTITIONER CAN RECOGNISE BEFORE SOLVING ANYTHING, AND THE EVIDENCE IS A MEASUREMENT REPORTED IN THIS PAPER RATHER THAN A PREFERENCE. THE RECOMMENDATION IS DELIBERATELY CONDITIONAL: THE TWO ROWS THAT PREFER THE NEURAL SOLVER ARE CASES WHERE THE CLASSICAL SCHEME *cannot* BE RUN OR *should not* BE RE-RUN, NOT CASES WHERE IT IS BEATEN ON ACCURACY.

Regime	Use	Deciding measurement
Single instance, $p \le 5$	**grid scheme**	faster *and* more accurate everywhere it runs: factor 5.6 in time at $p = 4$ as measured in NumPy, 114 compiled, and two to three orders at low p (Table II)
Control accuracy matters, not just value accuracy	**grid scheme**, and raise the reconstruction order	a cubic reconstruction on the same nodes raises the control-error order from 1.0 to 2.4 and cuts the control/value error ratio from 13.6 to 1.7 (Table VI)
$p \ge 8$, value function close to quadratic	**regression Monte Carlo**	it needs no grid either and is more accurate than the neural solver at every dimension tested, in seconds on a CPU (Table IX)
$p \ge 8$, value function not near-quadratic	**neural**, with the accuracy caveat	no classical method measured reaches this regime, and the neural solver's R^2 falls from 0.98 at $p = 8$ to 0.76 at $p = 20$ with no internal signal of the decay (Section VI-D)
A family of $\gtrsim 10^3$ problems	**learned operator**	the classical scheme must be re-solved per instance; an operator amortises after 1,000 to 4,365 parameter settings
Accuracy must be known, not assumed	**grid scheme**	the neural solver's accuracy decays from $R^2 = 0.98$ at $p = 8$ to 0.76 at $p = 20$ while its control-sign agreement stays near 0.7: no internal signal of the decay (Section VI-D)
Inputs will leave the training domain	**scale them explicitly**, whichever solver	an unscaled network's price sensitivity decays monotonically with price level, rank correlation -0.90 to -1.00, so any threshold on it yields an artefact (Section VII-C)

convolutions of [28], the branch/trunk construction of [55], and the discretisation-invariance arguments of [30] — towards models pretrained across families of equations and specified at query time by examples rather than by weights [56]. None of that is a solver for one instance, which is what this paper measures, and it is worth being explicit about the consequences rather than adding a row that would not be comparable. An operator is trained on a *family*, so the honest comparison is an amortised one: train once, evaluate on held-out members, and charge the classical methods a full re-solve per member. That comparison is well posed here — every member has a closed-form reference — and we have not run it, for a reason that is a property of this benchmark rather than a preference. The operator's input would be a low-dimensional parameter vector $(\lambda, \sigma, \eta, x_0)$, so the exercise would test whether a network can learn a smooth map from a handful of numbers. The methods already in this paper solve the same problem to a fraction of a percent in seconds: regression Monte Carlo at 0.4% relative error at $d = 1$, rising to 3.4% at $d = 20$, and multilevel Picard at 0.26% at $d = 20$. Operator learning earns its place when the input is a *function* — a volatility surface, an order-flow signal, a whole price path — which is the signalled and market-calibrated extension of Section III-A rather than the benchmark itself, and that is where we would place it.

One connection is worth stating because it is not obvious and it is a prediction. Proposition 1 is not about grid schemes: it says that any solver whose delivered object is a *derivative* of a reconstructed field inherits the reconstruction's order, and that a differentiation step caps it. An operator that outputs the value function and whose control is then obtained by differentiating the output — finite differences, or autograd of a network whose smoothness is set by its architecture — is in exactly that position, so the recommendation of Section VI-C transfers: if control accuracy matters, raise the order of the reconstruction before changing the solver. We state it as a prediction because we did not measure an operator on this benchmark, and the benchmark is released so that it can be tested against a closed-form reference.

C. Out-of-domain evaluation

A surrogate trained on a normalised state domain and evaluated in physical units fails quietly. A 4×64 $\tanh$ MLP trained on the HJB residual with terminal condition $V(S,T) = -\frac{1}{2}(S-25)^2$ on prices sampled from $[20, 30]$ only, then evaluated on the realised price levels of the nine instruments ($42 to $536), shows a learned price sensitivity $|\partial V/\partial S|$ that falls monotonically with price — from 3.36 at $42 to 0.062 at $536, a factor of about 54 — with rank correlations of -0.90, -0.98 and -1.00 across three seeds. The network saturates in proportion to how far its input lies beyond the training domain, so any threshold applied to this gradient inherits the ordering and produces a cross-sectional "finding" that admits a plausible microstructure story while being entirely an artefact of scale. Mapping the input back into the training domain removes the dependence ($+0.12$, -0.22, $+0.23$). What we add to the known out-of-domain failure of learned operators [57] is its *shape* in a control setting — saturation rather than divergence, so the damage survives every sanity check that looks for obviously broken output — and the procedural recommendation that follows: report per-channel input coverage for every evaluation set.

Limitations. (i) The cost half of the argument stops at $p \approx 5$, where the grid scheme stops being runnable at the resolution the accuracy target needs. That limit deserves one qualification, because the obvious objection to it is the wrong one. The restriction is not the CFL condition as such: we implemented the diffusion implicitly, by one-dimensional splitting, and it is stable and accurate at four times the explicit step (value R^2 still 0.999). But splitting is only first-order in time, so holding the temporal error below the spatial error still requires $dt \sim dx^2$ — the same rate the CFL condition imposes — and what implicitness buys is a constant factor of about four rather than a change in the scaling. The limitation is therefore the cost of controlling the temporal error, not stability; moving it needs a second-order time integrator, or sparse grids [58], which attack the N^d factor instead. (ii) The neural results use one architecture (4×64 $\tanh$), one optimiser and one schedule. Section VI-F measures what that choice is worth *within* the family — capacity and budget each move the boundary at $p = 4$ — but the sweep is still over MLPs at a fixed optimiser and fixed collocation, so it bounds the effect of size rather than of architecture as such. Techniques for high-dimensional PDEs [24], [25] could plausibly shift where the boundary sits, and so could architectural choices deliberately left out of scope here – wider residual blocks [59], or input feature maps that change the network's spectral bias and therefore which scales it can represent [60] – and we release the benchmark so that such claims can be tested against a closed-form reference. (iii) The separability effect is established as a phenomenon, not explained: we do not claim that symmetry *causes* the failure. (iv) Timings come from a single GPU and CPU, and the comparison is stated as GPU-for-neural versus CPU-for-grid, which favours the neural solver. (v) The control-accuracy result assumes a solver whose control is a finite difference of an interpolated value function; a scheme that solves for the control directly could narrow the gap, and we have not tested it. (vi) The model coefficients are synthetic rather than calibrated to an instrument, so no result here is a statement about how a particular asset trades. Section VI-H measures what calibration does to the conclusions, and the answer is not uniform: the *accuracy* comparison is unchanged, while the *cost* comparison is not, because a market-calibrated impact coefficient multiplies the explicit scheme's time-step requirement by 23–78$\times$ depending on the instrument. The speed claim therefore carries the scope stated in Section VI-H and not a general one. What the paper supplies no evidence about, under any calibration, is realised trading performance, and we offer none.

VIII. Conclusion

We compared neural and classical solvers for the HJB equation of multi-asset optimal execution against a closed-

form Riccati reference. The comparison produces a boundary rather than a winner, because the two families are not ranked along one axis but along three. Wherever a grid scheme can run, up to about five state dimensions, it is both faster and more accurate — by $5.6\times$ at $p = 4$ as measured in NumPy and $114\times$ with the same scheme compiled, which is also what flips the one dimension where the NumPy grid had been the slower method. Where it cannot run, the classical side is still not finished: regression Monte Carlo also needs no grid, and it beats the neural solver at every dimension where both were measured (R^2 of 0.999 at $p = 12$ and 0.977 at $p = 20$ against 0.927 and 0.757), in seconds on a CPU rather than minutes on a GPU. Multilevel Picard does better still where its hypothesis holds — 0.26% at $p = 20$ in 2.3 seconds — though at this paper's own calibration the Picard iteration it realises is not a contraction and it diverges, with the boundary measured. The neural solver's accuracy also decays with no internal signal that it is decaying, which is the property a practitioner cannot work around.

Measuring the *control*, which is what an HJB is for, says something more useful: value accuracy does not bound control accuracy, because what limits the control is the order of the operator that turns nodal values into a gradient. A multilinear reconstruction is first order, its exact gradient is also first order, and a cubic reconstruction on the same nodes reaches order 2.4 with the same probing step and 3.4 when that step is scaled with the mesh, cutting the control/value error ratio from 13.6 to 1.7 — worth 2.7 to 8.4 basis points of notional on the paper's own market calibration, and largest on the most liquid instrument. That improvement is free at the grid level, which is why the actionable recommendation is the second row of Table XIV: if control accuracy matters, change the reconstruction before changing the solver.

Two objections changed the picture without reversing it. A larger training budget recovers accuracy a smaller one loses, and we report the rescued run rather than the flattering comparison. And the field's preference for inference time is justified, because a classical solver must be re-solved for every instance — but the two framings answer different questions, one problem or a family of them, and have opposite answers. One term links the three limitations above: the maximised Hamiltonian $\frac{1}{2}\nabla V^\top B R^{-1} B^\top \nabla V$, whose coefficient is $1/\eta$, sets the grid's time-step limit, destabilises the explicit splitting recursion, and breaks multilevel Picard's contraction. The reconstruction question and the stiffness question are therefore the same question, and it is asked about the one coefficient the optimal-execution literature treats as a modelling constant. Both generalise: validate on the problem you actually have, with a reference you can trust.

Reproducibility

Code, the temporal data splits, the preprocessing manifest and every raw result file behind the tables are released at https://github.com/ridleyton/HJB-Solver-Comparison. The reference is checked against three independent closed forms, and the manuscript is built and page-checked by `src/build_paper.py`.

References

[1] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, 2008.

[2] O. Guéant, C.-A. Lehalle, and J. Fernandez-Tapia, "Dealing with the inventory risk: a solution to the market making problem," *Mathematics and Financial Economics*, vol. 7, no. 4, pp. 477–507, 2013.

[3] O. Guéant, "Closed-form approximations in multi-asset market making," *Applied Mathematical Finance*, 2016. volume and pages to confirm.

[4] Á. Cartea and S. Jaimungal, "Risk metrics and fine tuning of high-frequency trading strategies," *Mathematical Finance*, vol. 25, no. 3, pp. 576–611, 2015.

[5] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2000.

[6] Á. Cartea, S. Jaimungal, and J. Penalva, *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015.

[7] P. A. Forsyth and G. Labahn, "ϵ-monotone fourier methods for optimal stochastic control in finance," *arXiv preprint arXiv:1710.08450*, 2017.

[8] R. A. Howard, *Dynamic Programming and Markov Processes*. MIT Press, 1960.

[9] H. J. Kushner, "Numerical methods for stochastic control problems in continuous time," *SIAM Journal on Control and Optimization*, vol. 28, no. 5, pp. 999–1048, 1990.

[10] D. P. Bertsekas, *Dynamic Programming and Optimal Control, Vol. I*. Athena Scientific, 4th ed., 2012.

[11] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[12] J. Sirignano and K. Spiliopoulos, "Dgm: A deep learning algorithm for solving partial differential equations," *Journal of Computational Physics*, vol. 375, pp. 1339–1364, 2018.

[13] B. Yu and W. E, "The deep ritz method: A deep learning-based numerical algorithm for solving variational problems," *Communications in Mathematics and Statistics*, vol. 6, no. 1, pp. 1–12, 2018.

[14] W. E, J. Han, and A. Jentzen, "Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations," *Communications in Mathematics and Statistics*, vol. 5, no. 4, pp. 349–380, 2017.

[15] J. Han, A. Jentzen, and W. E, "Solving high-dimensional partial differential equations using deep learning," *Proceedings of the National Academy of Sciences*, vol. 115, no. 34, pp. 8505–8510, 2018.

[16] J. Han and W. E, "Deep learning approximation for stochastic control problems," *arXiv preprint arXiv:1611.07422*, 2016.

[17] C. Beck, W. E, and A. Jentzen, "Deep splitting method for parabolic pdes," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, pp. A3135–A3154, 2021.

[18] M. Fujii, A. Takahashi, and M. Takahashi, "A machine learning approach to Malliavin calculus with applications to stochastic optimal control," *Asia-Pacific Financial Markets*, vol. 26, no. 3, pp. 319–364, 2019.

[19] H. Pham, "Stochastic control problems with BSDEs and machine learning," *SIAM Journal on Financial Mathematics*, 2021. volume and pages to confirm.

[20] C. Huré, H. Pham, and X. Warin, "Deep backward schemes for high-dimensional nonlinear PDEs," *Mathematics of Computation*, vol. 89, no. 324, pp. 1547–1579, 2020.

[21] M. Germain, H. Pham, and X. Warin, "Approximation error analysis of some deep backward schemes for nonlinear PDEs," *SIAM Journal on Scientific Computing*, vol. 44, no. 1, pp. A28–A56, 2022.

[22] W. E, M. Hutzenthaler, and A. Jentzen, "On multilevel Picard iterations for high-dimensional nonlinear reaction-diffusion equations," *arXiv preprint arXiv:1908.01055*, 2019.

[23] M. Hutzenthaler, A. Jentzen, T. Kruse, and T. A. Nguyen, "Overcoming the curse of dimensionality in the numerical approximation of high-dimensional semilinear elliptic partial differential equations," *arXiv preprint arXiv:2003.00596*, 2020.

[24] X. Meng, Z. Li, D. Zhang, and G. E. Karniadakis, "Ppinn: Parareal physics-informed neural network for time-dependent pdes," *Computer Methods in Applied Mechanics and Engineering*, vol. 370, p. 113250, 2020.

[25] B. Moseley, A. Markham, and T. Nissen-Meyer, "Finite basis physics-informed neural networks (fbpinns): A scalable domain decomposition approach for solving differential equations," in *Advances in Neural Information Processing Systems*, 2021.

[26] R. Carmona and M. Laurière, "A machine learning framework for solving high-dimensional mean field game and mean field control problems," *Proceedings of the National Academy of Sciences*, vol. 117, no. 17, pp. 9183–9193, 2020.

[27] H. Bühler, L. Gonon, J. Teichmann, and B. Wood, "Deep hedging," *Quantitative Finance*, vol. 19, no. 8, pp. 1271–1291, 2019.

[28] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," in *International Conference on Learning Representations (ICLR)*, 2021.

[29] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Neural operator: Learning maps between function spaces with applications to pdes," *Journal of Machine Learning Research*, vol. 24, no. 89, pp. 1–97, 2023.

[30] B. Raonić, R. Molinaro, T. De Ryck, T. Rohner, F. Bartolucci, R. Alaifari, S. Mishra, and E. de Bézenac, "Convolutional neural operators for robust and accurate learning of pdes," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[31] H. Wu, H. Luo, H. Wang, J. Wang, and M. Long, "Transolver: A fast transformer solver for pdes on general geometries," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, vol. 235 of *PMLR*, 2024.

[32] X. Han, F. Hou, and H. Qin, "Ugrid: An efficient-and-rigorous neural multigrid solver for linear pdes," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, vol. 235 of *PMLR*, 2024.

[33] P. Wei, M. Liu, J. Cen, Z. Zhou, L. Chen, and Q. Zou, "Pdenneval: A comprehensive evaluation of neural network methods for solving pdes," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[34] F. A. Longstaff and E. S. Schwartz, "Valuing American options by simulation: a simple least-squares approach," *The Review of Financial Studies*, vol. 14, no. 1, pp. 113–147, 2001.

[35] P. Glasserman, *Monte Carlo Methods in Financial Engineering*. Springer, 2003.

[36] I. Kharroubi, N. Langrené, and H. Pham, "A numerical algorithm for fully nonlinear HJB equations: an approach by control randomization," *Monte Carlo Methods and Applications*, vol. 20, no. 2, pp. 145–165, 2014.

[37] C. Bender and J. Steiner, "Least-squares Monte Carlo for BSDEs," in *Monte Carlo and Quasi-Monte Carlo Methods 2010*, pp. 199–217, Springer, 2012.

[38] T. Xiao, R. Yang, Y. Cheng, and J. Suo, "Shop: A deep learning framework for solving high-order partial differential equations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 16032–16039, 2024.

[39] R. Roll, "A simple implicit measure of the effective bid-ask spread in an efficient market," *The Journal of Finance*, vol. 39, no. 4, pp. 1127–1139, 1984.

[40] S. A. Corwin and P. Schultz, "A simple way to estimate bid-ask spreads from daily high and low prices," *The Journal of Finance*, vol. 67, no. 2, pp. 719–760, 2012.

[41] G. Barles and P. E. Souganidis, "Convergence of approximation schemes for fully nonlinear second order equations," *Asymptotic Analysis*, vol. 4, no. 3, pp. 271–283, 1991.

[42] M. G. Crandall and P.-L. Lions, "Viscosity solutions of Hamilton–Jacobi equations," *Transactions of the American Mathematical Society*, vol. 277, no. 1, pp. 1–42, 1983.

[43] A. Harten, B. Engquist, S. Osher, and S. R. Chakravarthy, "Uniformly high order accurate essentially non-oscillatory schemes, III," *Journal of Computational Physics*, vol. 71, no. 2, pp. 231–303, 1987.

[44] C.-W. Shu, "Essentially non-oscillatory and weighted essentially non-oscillatory schemes for hyperbolic conservation laws," *NASA/CR-97-206253, ICASE Report No. 97-65*, 1998.

[45] X.-D. Liu, S. Osher, and T. Chan, "Weighted essentially non-oscillatory schemes," *Journal of Computational Physics*, vol. 115, no. 1, pp. 200–212, 1994.

[46] M. Falcone and R. Ferretti, "Semi-Lagrangian schemes for Hamilton–Jacobi equations, discrete representation formulae and Godunov methods," *Journal of Computational Physics*, vol. 175, no. 2, pp. 559–575, 2002.

[47] O. C. Zienkiewicz and J. Z. Zhu, "The superconvergent patch recovery and a posteriori error estimates. Part 1: The recovery technique," *International Journal for Numerical Methods in Engineering*, vol. 33, no. 7, pp. 1339–1364, 1992.

[48] B. Cockburn, M. Luskin, C.-W. Shu, and E. Süli, "Enhanced accuracy by post-processing for finite element methods for hyperbolic equations," *Mathematics of Computation*, vol. 72, no. 242, pp. 577–606, 2003.

[49] R. S. Falk, "Approximation of a class of optimal control problems with order of convergence estimates," *Journal of Mathematical Analysis and Applications*, vol. 44, no. 1, pp. 28–47, 1973.

[50] M. Hinze, R. Pinnau, M. Ulbrich, and S. Ulbrich, *Optimization with PDE Constraints*. Springer, 2009.

[51] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[52] M. Hutzenthaler, A. Jentzen, T. Kruse, and T. A. Nguyen, "Multilevel Picard approximations for high-dimensional semilinear second-order PDEs with Lipschitz nonlinearities," *arXiv preprint arXiv:2009.02484*, 2020.

[53] A. Neufeld and T. A. Wu, "Multilevel Picard algorithm for general semilinear parabolic PDEs with gradient-dependent nonlinearities," *arXiv preprint arXiv:2310.12545*, 2023.

[54] S. Wang, H. Wang, and P. Perdikaris, "Physics-informed neural operator for learning partial differential equations," *ACM/IMS Journal of Data Science*, 2021. arXiv:2111.03794.

[55] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, 2021.

[56] L. Yang and S. J. Osher, "PDE generalization of in-context operator networks: a study on 1D scalar nonlinear conservation laws," *Journal of Computational Physics*, vol. 519, p. 113379, 2024.

[57] S. C. Mouli, D. C. Maddix, S. Alizadeh, G. Gupta, A. Stuart, M. W. Mahoney, and Y. Wang, "Using uncertainty quantification to characterize and improve out-of-domain learning for pdes," *arXiv preprint arXiv:2403.10642*, 2024.

[58] H.-J. Bungartz and M. Griebel, "Sparse grids," *Acta Numerica*, vol. 13, pp. 147–269, 2004.

[59] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *British Machine Vision Conference (BMVC)*, 2016.

[60] S. Wang, H. Wang, and P. Perdikaris, "On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 384, p. 113938, 2021.